```
/* ==================== 文件：src/main.js ==================== */
import { createApp } from 'vue'
import ElementPlus from 'element-plus'
import 'element-plus/dist/index.css'
import zhCn from 'element-plus/es/locale/lang/zh-cn'
import App from './App.vue'
import './style.css'

// 创建 Vue 应用，全局注册 Element Plus（中文语言包）
createApp(App)
  .use(ElementPlus, { locale: zhCn })
  .mount('#app')
/* ==================== 文件：src/App.vue ==================== */
<!--
  ================================================================
  辅助学习系统 · 根组件（V6.0）
  ----------------------------------------------------------------
  系统由三大板块组成：
    ① 游戏：游戏一（单机政策模拟 Game1View）+ 游戏二（多人博弈）
    ② 知识学习：章节学习 / 题库 / 错题本 / 知识图谱 …
    ③ AI：学生画像与学习建议（StudentProfileView）+ AI 助手 + 教师端

  视图切换方式：状态机（不引入 vue-router），每个视图一个字符串 key。
  ----------------------------------------------------------------
  责任边界：
    · App 只做"视图编排 + 全局事件汇聚"，不承载业务逻辑；
    · 游戏一的内核事件由 Game1View emit 上来，这里统一转成
      两条数据：learning_event（过程，走 reportEvent）
                game1_session（结果，走 saveGame1Session）；
    · 页面卸载/隐藏时统一 flush 事件队列，避免丢最后一批操作。
================================================================ -->
<script setup>
import { ref, onMounted, onBeforeUnmount } from 'vue'
import HomeView from './views/HomeView.vue'
import Chapter11View from './views/learning/Chapter11View.vue'
import ExerciseView from './views/learning/ExerciseView.vue'
import WrongBookView from './views/learning/WrongBookView.vue'
import ScheduleView from './views/learning/ScheduleView.vue'
import StatsView from './views/learning/StatsView.vue'
import ResourceView from './views/learning/ResourceView.vue'
import CommunityView from './views/learning/CommunityView.vue'
import KnowledgeGraphView from './views/learning/KnowledgeGraphView.vue'
import AiAssistantView from './views/ai/AiAssistantView.vue'
import StudentProfileView from './views/ai/StudentProfileView.vue'
import LoginView from './views/common/LoginView.vue'
import Game1View from './views/game/Game1View.vue'
import Game2LobbyView from './views/game/Game2LobbyView.vue'
import Game2RoomView from './views/game/Game2RoomView.vue'
import TeacherView from './views/teacher/TeacherView.vue'
import { authStore } from '@/store/auth'
```

```
import { syncOnLogin } from '@/store/learning'
import { reportEvent, installEventFlusher, flushEvents } from '@/api/event'
import { saveGame1Session } from '@/api/game1'
import { createGame1Recorder, withNickname } from '@/utils/game1Recorder'

/** 当前视图标识 */
const currentView = ref('home')
/** 当前打开的章节 id */
const currentChapter = ref(null)
/** 当前多人博弈房间码 */
const currentGameCode = ref(null)

/** 游戏一记录器（跨事件累计本局统计） */
const game1Recorder = createGame1Recorder()

/* ================= 页面导航 ================= */

/** 打开章节学习页 */
function openChapter(chapterId) {
  currentChapter.value = chapterId || 'ch11'
  currentView.value = 'chapter'
  window.scrollTo(0, 0)
}

/** 打开功能页面（首页宫格与各页"去完成"入口都走这里） */
function openPage(page) {
  /* 章节学习依赖 chapterId，这里补一个默认值，避免从宫格进入时白屏 */
  if (page === 'chapter') return openChapter(currentChapter.value || 'ch11')
  currentView.value = page
  window.scrollTo(0, 0)
}

/** 进入多人博弈房间（大厅 → 主界面） */
function enterGame(code) {
  currentGameCode.value = code
  currentView.value = 'game-room'
  window.scrollTo(0, 0)
}

/** 统一返回首页 */
function backHome() {
  /* 返回前把未上报的事件发出去（游戏中途退出也不丢数据） */
  flushEvents(true)
  currentView.value = 'home'
  currentChapter.value = null
  currentGameCode.value = null
  window.scrollTo(0, 0)
}

/* ================= 游戏一数据上报 ================= */
```

```
/** 学生昵称（匿名时给空串，由组件显示"匿名学生"） */
function currentNickname() {
  return (authStore.state.user && (authStore.state.user.displayName || authStore.state.user.user
    name)) || ''
}

/**
 * 接收 Game1View 抛上来的内核事件
 * @param {{type:string, payload:object}} evt
 */
function onGame1Event(evt) {
  if (!evt || !evt.type) return
  /* ① 过程：写事件流（AI 画像读取"游戏中的操作"就靠这里） */
  reportEvent('game1', evt.type, evt.payload)
  /* ② 结果：结算时把整局快照写 game1_session */
  const session = game1Recorder.handle(evt.type, evt.payload)
  if (session) {
    saveGame1Session(withNickname(session, currentNickname())).catch(() => {
      /* 后端不可用时静默失败：不影响学生继续游戏 */
    })
  }
}

/* ================= 生命周期 ================= */

onMounted(async () => {
  await authStore.checkToken()
  syncOnLogin()
  installEventFlusher()
})

onBeforeUnmount(() => {
  flushEvents(true)
})
</script>

<template>
  <!-- 首页 -->
  <HomeView v-if="currentView === 'home'" @open-chapter="openChapter" @open-page="openPage" />

  <!-- ① 游戏板块 -->
  <Game1View
    v-else-if="currentView === 'game1'"
    @back="backHome"
    @event="onGame1Event"
  />
  <Game2LobbyView
    v-else-if="currentView === 'game2'"
    @back="backHome"
```

```
    @enter="enterGame"
  />
  <Game2RoomView
    v-else-if="currentView === 'game-room'"
    :code="currentGameCode"
    @back="backHome"
    @again="() => currentView = 'game2'"
  />

  <!-- ② 知识学习板块 -->
  <Chapter11View v-else-if="currentView === 'chapter'" @back="backHome" />
  <ExerciseView v-else-if="currentView === 'exercise'" @back="backHome" />
  <KnowledgeGraphView
    v-else-if="currentView === 'knowledge'"
    @back="backHome"
    @open-page="openPage"
    @open-chapter="openChapter"
  />
  <WrongBookView v-else-if="currentView === 'wrongbook'" @back="backHome" />
  <ScheduleView v-else-if="currentView === 'schedule'" @back="backHome" />
  <StatsView v-else-if="currentView === 'stats'" @back="backHome" />
  <ResourceView v-else-if="currentView === 'resource'" @back="backHome" />
  <CommunityView v-else-if="currentView === 'community'" @back="backHome" />

  <!-- ③ AI 板块 -->
  <StudentProfileView
    v-else-if="currentView === 'profile'"
    @back="backHome"
    @open-page="openPage"
    @open-chapter="openChapter"
  />
  <AiAssistantView v-else-if="currentView === 'ai'" @back="backHome" />
  <TeacherView v-else-if="currentView === 'teacher'" @back="backHome" @open-page="openPage" />

  <!-- 账号 -->
  <LoginView v-else-if="currentView === 'login'" @back="backHome" />
</template>
/* ==================== 文件：src/views/HomeView.vue ==================== */
<!--
  辅助学习系统 - 首页
  V5.0：章节选择 + 八大功能模块导航
-->
<template>
  <div class="home-container">

    <!-- 顶部标题 -->
    <div class="hero">
      <!-- 右上角登录区：未登录显示入口，已登录显示昵称与退出 -->
      <div class="auth-zone">
        <template v-if="authStore.isLoggedIn()">
```

```
      <span class="auth-user">👋 {{ authStore.state.user.displayName || authStore.state.user
.username }}</span>
      <el-button size="small" round @click="authStore.logout()">退出</el-button>
    </template>
    <el-button v-else type="warning" size="small" round @click="$emit('open-page', 'login')"
>
      登录 / 注册
    </el-button>
  </div>

  <h1 class="hero-title">宏观经济学智能学习系统</h1>
  <p class="hero-sub">
    🎮 双游戏仿真 × 📖 知识图谱学习 × 🤖 AI 学习画像与个性化建议 × 👩‍🏫 教师端学情看板<br/>
    系统读取你在两游戏中的操作与学习行为，用强化学习神经网络为你规划下一步。
  </p>
</div>

<!-- 功能导航宫格 -->
<div class="feature-grid">
  <div
    v-for="f in features"
    :key="f.page"
    class="feature-card"
    :style="{ '--c1': f.c1, '--c2': f.c2 }"
    @click="$emit('open-page', f.page)"
  >
    <div class="f-icon">{{ f.icon }}</div>
    <div class="f-name">{{ f.name }}</div>
    <div class="f-desc">{{ f.desc }}</div>
    <span v-if="f.hot" class="f-hot">🔥 {{ f.hot }}</span>
  </div>
</div>

<!-- 章节列表 -->
<h3 class="sec-title">📚 章节学习</h3>
<div class="chapter-grid">

  <!-- 第十一章（已开放） -->
  <el-card class="chapter-card open" shadow="hover">
    <div class="ch-no">第十一章</div>
    <div class="ch-title">国民收入的决定：IS-LM 模型</div>
    <div class="ch-desc">
      IS 曲线（产品市场均衡）、LM 曲线（货币市场均衡）、IS-LM 共同均衡与政策运用、本章评析。
    </div>
    <div class="ch-tags">
      <el-tag size="small" type="primary">4 个小节</el-tag>
      <el-tag size="small" type="success">3 个交互实验</el-tag>
      <el-tag size="small" type="warning">8 道习题</el-tag>
      <el-tag size="small" type="danger">仿真验证</el-tag>
    </div>
```

```
        <el-button type="primary" size="large" class="ch-btn" @click="$emit('open-chapter', 'ch1
    1')">
          📖 开始学习
        </el-button>
      </el-card>

      <!-- 占位章节（建设中） -->
      <el-card v-for="n in 3" :key="n" class="chapter-card locked" shadow="never">
        <div class="ch-no">第{{ ['十二', '十三', '十四'][n - 1] }}章</div>
        <div class="ch-title locked-text">敬请期待</div>
        <div class="ch-desc">该章节正在建设中……</div>
        <el-button size="large" class="ch-btn" disabled>🔒 未开放</el-button>
      </el-card>

    </div>
  </div>
</template>

<script setup>
import { authStore } from '@/store/auth'

defineEmits(['open-chapter', 'open-page'])

/* 功能模块配置
 * 三大板块顺序：① 双游戏仿真 ② 知识学习 ③ AI 能力，教师端置尾
 * hot 字段用于角标：'核心玩法' / 'AI 驱动' 等 */
const features = [
  { page: 'game1',     icon: '🕹️', name: '游戏一 · 经济舵手', desc: '14 个季度、44 张政策卡，
    亲手操作财政/货币/产业/贸易工具推动四条曲线', c1: '#ff9a44', c2: '#fc6076', hot: '政策模拟'
    },
  { page: 'game2',     icon: '🎮', name: '游戏二 · 多人博弈', desc: '4 人分饰政府（含央行）/银行
    /企业/消费者，实时博弈推动 IS-LM', c1: '#ff512f', c2: '#dd2476', hot: '小组对抗' },
  { page: 'chapter',   icon: '📖', name: '章节学习',   desc: '知识点讲解 + 图示推导 + 交互仿真实
    验', c1: '#667eea', c2: '#764ba2' },
  { page: 'knowledge', icon: '⚛️', name: '知识图谱',   desc: '25 个知识节点 + 先修关系，按你的
    掌握度着色，AI 规划学习前沿', c1: '#43e97b', c2: '#38f9d7' },
  { page: 'exercise',  icon: '📝', name: '题库练习',   desc: '专项刷题 / 模拟测试 / 限时自测，
    自动批改', c1: '#f093fb', c2: '#f5576c' },
  { page: 'profile',   icon: '📌', name: '我的学习画像', desc: 'AI 读取两游戏操作与学习行为，生
    成六维画像与针对性建议', c1: '#00c6ff', c2: '#0072ff', hot: 'AI 驱动' },
  { page: 'ai',        icon: '🤖', name: 'AI 学习助手', desc: '引导式答疑（不直接给答案）+ 自动
    生成复习资料', c1: '#43e97b', c2: '#38f9d7' },
  { page: 'wrongbook', icon: '📕', name: '智能错题本', desc: '错题自动收纳、标注原因、按知识点重
    做', c1: '#fa709a', c2: '#fee140' },
  { page: 'stats',     icon: '📊', name: '学习数据统计', desc: '刷题时长、掌握度图表、学习周报、
    薄弱板块', c1: '#a8edea', c2: '#536976' },
  { page: 'schedule',  icon: '🗓️', name: '课程与日程', desc: '课程表、学习日历、作业考试倒计时
    、待办计划', c1: '#30cfd0', c2: '#330867' },
  { page: 'resource',  icon: '📂', name: '学习资源库', desc: '课件 / 笔记 / 试卷 / 提纲，上传下
    载与搜索', c1: '#ff9a9e', c2: '#a18cd1' },
```

```
  { page: 'community', icon: '💬', name: '问答社区',  desc: '课程疑问互助答疑，热门问题沉淀高频
    知识库', c1: '#4facfe', c2: '#00f2fe' },
  { page: 'teacher',  icon: '👩‍🏫', name: '教师端 · 学情看板', desc: '班级掌握度、风险分级、学
    生画像与 AI 教学建议（需教师账号）', c1: '#606c88', c2: '#3f4c6b' }
]
</script>

<style scoped>
.home-container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 30px 20px;
  min-height: 100vh;
  background: linear-gradient(135deg, #f5f7fa 0%, #e4e8f0 100%);
  font-family: 'Microsoft YaHei', 'PingFang SC', sans-serif;
}

.hero {
  position: relative;
  text-align: center;
  padding: 40px 20px;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: #fff;
  border-radius: 14px;
  margin-bottom: 26px;
  box-shadow: 0 6px 20px rgba(102, 126, 234, 0.35);
}
/* 右上角登录区 */
.auth-zone {
  position: absolute;
  top: 14px;
  right: 16px;
  display: flex;
  align-items: center;
  gap: 8px;
}
.auth-user { font-size: 13px; opacity: 0.95; }
.hero-title {
  margin: 0 0 12px;
  font-size: 32px;
  letter-spacing: 2px;
}
.hero-sub {
  margin: 0;
  font-size: 14px;
  opacity: 0.92;
  line-height: 1.9;
}

/* 功能宫格 */
```

```
.feature-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(260px, 1fr));
  gap: 16px;
  margin-bottom: 30px;
}
.feature-card {
  background: #fff;
  border-radius: 12px;
  padding: 18px;
  cursor: pointer;
  border: 1px solid #ebeef5;
  transition: transform 0.22s, box-shadow 0.22s;
  position: relative;
  overflow: hidden;
}
.feature-card::before {
  content: '';
  position: absolute;
  left: 0; top: 0; bottom: 0;
  width: 4px;
  background: linear-gradient(180deg, var(--c1), var(--c2));
}
.feature-card:hover {
  transform: translateY(-4px);
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.1);
}
.f-icon { font-size: 30px; margin-bottom: 8px; }
.f-name { font-size: 16px; font-weight: 700; color: #303133; margin-bottom: 6px; }
.f-desc { font-size: 12.5px; color: #909399; line-height: 1.6; }
.f-hot {
  position: absolute; top: 12px; right: 12px;
  font-size: 11px; color: #fff; background: linear-gradient(135deg, #ff512f, #dd2476);
  padding: 2px 8px; border-radius: 10px;
}

/* 章节列表 */
.sec-title {
  color: #303133;
  margin: 0 0 14px;
}
.chapter-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(320px, 1fr));
  gap: 20px;
  margin-bottom: 30px;
}
.chapter-card {
  border-radius: 12px;
  display: flex;
```

```
  flex-direction: column;
  transition: transform 0.25s;
}
.chapter-card.open {
  border: 2px solid #667eea;
}
.chapter-card.open:hover {
  transform: translateY(-4px);
}
.chapter-card.locked {
  opacity: 0.65;
  border: 2px dashed #dcdfe6;
}
.ch-no {
  font-size: 13px;
  color: #909399;
  letter-spacing: 3px;
  margin-bottom: 6px;
}
.ch-title {
  font-size: 19px;
  font-weight: 700;
  color: #303133;
  margin-bottom: 10px;
}
.ch-title.locked-text {
  color: #c0c4cc;
}
.ch-desc {
  font-size: 13px;
  color: #606266;
  line-height: 1.7;
  min-height: 66px;
  margin-bottom: 12px;
}
.ch-tags {
  display: flex;
  gap: 6px;
  flex-wrap: wrap;
  margin-bottom: 16px;
}
.ch-btn {
  width: 100%;
  margin-top: auto;
}
</style>
/* ==================== 文件：src/views/game/Game1View.vue ==================== */
<!--
  游戏一 · 经济舵手 Macro Pilot
  ------------------------------------------------------------
```

```
  来源：D:\HZYJ\game1（纯 HTML/CSS/JS 单机教学 Demo）
  移植要点：
    1. template 内逐字保留原 index.html <body> 的全部 DOM 结构与 id / class / 文案，
       最外层套 .game1-view（供 mountGame1 做 root 作用域查询），
       内层 .game1-stage 承载原深色游戏舞台（外层仍是站点的浅色底）；
    2. 不用 scoped：原 DOM 大量由 JS 动态生成，scoped 属性选择器命中不到，
       样式隔离改由 src/game1/style.css 的 .game1-view 前缀完成；
    3. 事件双向：mountGame1 的 onEvent 既通过 emit('event') 抛给父组件，
       也在 game1.finish 时额外走 emit('finish')，便于父组件做结算处理。
-->
<script setup>
import { ref, onMounted, onBeforeUnmount } from 'vue'
import { mountGame1 } from '../../game1/index.js'
/* 原 style.css 已整体加 .game1-view 前缀，这里直接引入（非 scoped，保证动态 DOM 也能命中） */
import '../../game1/style.css'

const emit = defineEmits(['back', 'event', 'finish'])

/* 游戏容器（mountGame1 的 root，所有 DOM 查询的作用域根） */
const rootEl = ref(null)
/* 右上角显示的学生昵称 */
const studentName = ref('匿名学生')

/* mountGame1 返回的实例（保存以便卸载时 destroy） */
let instance = null

/**
 * 读取当前学生档案
 * islm-auth 结构：{ token, user: { id, username, displayName } }
 * 解析失败 / 未登录 → 匿名学生
 */
function readProfile() {
  try {
    const raw = localStorage.getItem('islm_auth')
    if (!raw) return { displayName: '匿名学生' }
    const auth = JSON.parse(raw)
    const displayName = auth && auth.user && auth.user.displayName
    return { displayName: displayName || '匿名学生' }
  } catch (e) {
    return { displayName: '匿名学生' }
  }
}

/**
 * 事件转发：游戏内核上报 → 组件 emit
 * @param {string} type    game1.start / game1.policy / game1.quiz / game1.quarter / game1.wrong
    book / game1.finish
 * @param {object} payload 纯 JSON 载荷
 */
function handleEvent(type, payload) {
```

```
  emit('event', { type, payload })
  if (type === 'game1.finish') emit('finish', payload)
}

onMounted(() => {
  const profile = readProfile()
  studentName.value = profile.displayName
  instance = mountGame1({
    root: rootEl.value,
    onEvent: handleEvent,
    profile,
  })
})

/* 卸载：彻底清理监听 / 定时器 / 动画，保证再次进入不重复绑定 */
onBeforeUnmount(() => {
  if (instance) {
    instance.destroy()
    instance = null
  }
})

/* 父组件如需读取当前局面（只读快照），可直接调用 */
defineExpose({
  getState: () => (instance ? instance.getState() : null),
})
</script>

<template>
  <div class="game1-view" ref="rootEl">

    <!-- 顶部返回栏（开始界面同样可见） -->
    <header class="g1-return-bar">
      <button class="g1-back" type="button" @click="$emit('back')">← 返回首页</button>
      <div class="g1-titles">
        <h1 class="g1-title">🎮 游戏一 · 经济舵手 Macro Pilot</h1>
        <p class="g1-sub">宏观经济政策模拟 · 14 个季度 · 44 张政策卡</p>
      </div>
      <div class="g1-user">👤 {{ studentName }}</div>
    </header>

    <!-- 深色游戏舞台：原 index.html <body> 的全部结构 -->
    <div class="game1-stage">

      <!-- ===================== 开始界面 ===================== -->
      <div id="start-screen">
        <div class="start-box paper-note">
          <h1>🛞 经济舵手 <span class="en">Macro Pilot</span></h1>
          <p class="subtitle">—— 宏观经济政策模拟 · 经济学本科生教学 Demo ——</p>
          <ul class="feature-list">
```

```
        <li>🎓 两种模式：学习模式（先做题再出手） / 沙盘模式（免做题自由试验）</li>
        <li>📉 连续时间线：2023 Q4 → 2027 Q1，共 14 个季度，完整经历一轮经济周期</li>
        <li>🃏 44 张政策卡牌：财政 / 货币 / 产业（中国特供）/ 贸易四大工具箱</li>
        <li>🔍 传导演示：政策落地即播放六步链条动画——冲击注入→曲线移动→均衡重解→物价→就业→部门反馈，数值实时滚动</li>
        <li>📊 动态模型：IS-LM、AD-AS、菲利普斯曲线实时动画推演</li>
        <li>🧠 因素链条：点击传导链节点，弹出公式推导与教材出处</li>
      </ul>
      <div class="mode-row">
        <button class="mode-btn" id="btn-learn">
          <span class="mb-icon">🎓</span>
          <b>学习模式</b>
          <span class="mb-desc">每张政策先做题，答对全额生效、答错减半；传导演示附公式推导</span>
        </button>
        <button class="mode-btn" id="btn-sandbox">
          <span class="mb-icon">🧪</span>
          <b>沙盘模式</b>
          <span class="mb-desc">免做题、政策即时全额生效，每季可执行 5 项，自由试验模型</span>
        </button>
      </div>
      <p class="tip">建议使用 Chrome / Edge 浏览器，窗口宽度 ≥ 1440px</p>
    </div>
  </div>

  <!-- ==================== 主界面 ==================== -->
  <div id="app" class="hidden">

    <!-- ① 顶部：全局导航与核心指标盘 -->
    <header id="topbar">
      <div class="logo">🛞 <b>经济舵手</b><span class="en">Macro Pilot</span></div>
      <div id="kpi-cards"><!-- KPI 卡片由 JS 生成 --></div>
      <div class="time-controls">
        <div id="quarter-label">2023 Q4</div>
        <button id="btn-anim-speed" title="切换动画速度">🐢 慢速</button>
        <button id="btn-wrongbook" title="错题本">📒 错题本</button>
        <button id="btn-end-quarter">结束本季度 ▶</button>
      </div>
    </header>

    <div id="main-grid">

      <!-- ② 左侧：政策工具箱 -->
      <aside id="toolbox">
        <h2>🧰 政策工具箱</h2>
        <div id="drawer-container"><!-- 四个抽屉由 JS 生成 --></div>
        <div id="pending-box">
          <h3>本季度已执行政策 <span id="pending-count">0 / 3</span></h3>
          <ul id="pending-list"></ul>
```

```
      </div>
    </aside>

    <!-- ③ 中央：动态经济全景与图表区 -->
    <section id="center">
      <div id="city-map" class="panel"><!-- 手绘城市地图由 JS 生成 --></div>
      <div id="timeline" class="panel"><!-- 时间轴由 JS 生成 --></div>
      <div id="chart-row">
        <div class="chart-box"><h3>GDP 构成堆积（万亿）</h3><svg id="chart-gdp"></svg></di
v>
        <div class="chart-box"><h3>IS-LM 模型</h3><svg id="chart-islm"></svg></div>
        <div class="chart-box"><h3>AD-AS 模型</h3><svg id="chart-adas"></svg></div>
        <div class="chart-box"><h3>菲利普斯曲线</h3><svg id="chart-phillips"></svg></div>
      </div>
    </section>

    <!-- ④ 右侧：资讯与反馈区 -->
    <aside id="feed">
      <div class="panel" id="news-panel"><h3>📰 新闻与事件</h3><ul id="news-list"></ul></d
iv>
      <div class="panel" id="firm-panel"><h3>🏭 企业动态</h3><ul id="firm-list"></ul></div
>
      <div class="panel" id="expect-panel">
        <h3>🔮 公众预期</h3>
        <div id="expect-content"></div>
      </div>
      <div class="panel" id="advisor-panel">
        <h3>👩‍🏫 经济顾问</h3>
        <div id="advisor-note" class="paper-note small"></div>
      </div>
    </aside>

    <!-- ⑤ 底部：教学闭环与结算区 -->
    <footer id="bottom">
      <div class="panel" id="chain-panel">
        <h3>🔗 因素链条（点击节点看公式推导）
          <select id="chain-category">
            <option value="fiscal">财政政策传导</option>
            <option value="monetary">货币政策传导</option>
            <option value="trade">贸易政策传导</option>
          </select>
        </h3>
        <div id="chain-container"></div>
      </div>
      <div class="panel" id="advice-panel"><h3>📌 学习建议</h3><ul id="advice-list"></ul><
/div>
      <div class="panel" id="radar-panel"><h3>🎯 本局评分</h3><svg id="radar-svg"></svg></
div>
      <div class="panel" id="achv-panel"><h3>🏅 成就徽章</h3><ul id="achv-list"></ul></div
>
```

```
      </footer>
    </div>
  </div>

  <!-- ==================== © 弹窗层 ==================== -->

  <!-- 做题弹窗（手绘便签纸风格） -->
  <div id="quiz-modal" class="modal-layer hidden">
    <div class="paper-note quiz-paper">
      <div class="quiz-head"><span id="quiz-card-name"></span><span id="quiz-stamp"></span><
/div>
      <div id="quiz-question"></div>
      <div id="quiz-options"></div>
      <div id="quiz-explain" class="hidden"></div>
      <div class="quiz-foot">
        <span id="quiz-chapter"></span>
        <button id="quiz-close" class="hidden">关闭</button>
      </div>
    </div>
  </div>

  <!-- 知识卡弹窗（因素链条节点） -->
  <div id="knowledge-modal" class="modal-layer hidden">
    <div class="paper-note quiz-paper">
      <h3 id="know-title"></h3>
      <div id="know-formula" class="formula-box"></div>
      <div id="know-text"></div>
      <div class="quiz-foot"><span id="know-chapter"></span><button id="know-close">关闭</bu
tton></div>
    </div>
  </div>

  <!-- 事件新闻卡弹窗 -->
  <div id="event-modal" class="modal-layer hidden">
    <div class="paper-note newspaper">
      <h3 id="event-title"></h3>
      <p id="event-desc"></p>
      <p id="event-effect"></p>
      <button id="event-close">知道了</button>
    </div>
  </div>

  <!-- 错题本弹窗 -->
  <div id="wrongbook-modal" class="modal-layer hidden">
    <div class="paper-note quiz-paper">
      <h3>📒 错题本</h3>
      <div id="wrongbook-list"></div>
      <div class="quiz-foot"><span>答对同类新题可移出</span><button id="wrongbook-close">关
闭</button></div>
    </div>
```

```
      </div>

      <!-- 结算弹窗 -->
      <div id="final-modal" class="modal-layer hidden">
        <div class="paper-note quiz-paper final-paper">
          <h3>🏁 14 个季度执舵结束！</h3>
          <div id="final-summary"></div>
          <div class="quiz-foot"><button id="final-restart">再来一局 🔁</button></div>
        </div>
      </div>

      <!-- 传导演示弹窗（六步链条动画） -->
      <div id="transmission-modal" class="modal-layer hidden">
        <div class="paper-note trans-paper">
          <h3 id="trans-title">🎯 传导演示</h3>
          <div id="trans-chain"></div>
          <div class="trans-foot">
            <span class="trans-hint">每个节点即时重解均衡，数值实时滚动</span>
            <div>
              <button id="trans-skip">跳过动画 ⏩</button>
              <button id="trans-done" class="hidden">确认效果 ▶</button>
            </div>
          </div>
        </div>
      </div>

      <!-- 轻提示层 -->
      <div id="toast-layer"></div>
    </div>
  </div>
</template>

<style>
/* ============================================================
   页面外壳：浅色底 + 最大宽度 1600px 居中，嵌入现有站点
   （游戏本身的深色舞台在 .game1-stage 内，样式见 src/game1/style.css）
   ============================================================ */
.game1-view {
  max-width: 1600px;
  margin: 0 auto;
  padding: 0 16px 18px;
  background: #ffffff;
  box-sizing: border-box;
}

/* ---------- 顶部返回栏 ---------- */
.g1-return-bar {
  display: flex;
  align-items: center;
  gap: 16px;
```

```
  padding: 12px 4px;
  margin-bottom: 8px;
  border-bottom: 1px solid #e6eaf2;
  color: #1f2937;
  font-family: "Microsoft YaHei", "PingFang SC", sans-serif;
}
.g1-back {
  flex: 0 0 auto;
  padding: 7px 14px;
  font-size: 14px;
  color: #2b6cb0;
  background: #f1f6fd;
  border: 1px solid #cfe0f5;
  border-radius: 8px;
  cursor: pointer;
  transition: all .15s;
}
.g1-back:hover {
  background: #e3eefb;
  border-color: #9fc3ea;
}
.g1-titles { flex: 1; min-width: 0; }
.g1-title {
  font-size: 20px;
  font-weight: 700;
  color: #16233f;
  margin: 0;
  line-height: 1.3;
}
.g1-sub {
  margin: 2px 0 0;
  font-size: 12.5px;
  color: #6b7a90;
}
.g1-user {
  flex: 0 0 auto;
  font-size: 13px;
  color: #334155;
  background: #f5f7fa;
  border: 1px solid #e2e8f0;
  border-radius: 999px;
  padding: 5px 14px;
  white-space: nowrap;
}

/* ---------- 游戏舞台容器 ----------
   给出固定高度并裁掉圆角外的深色底;
   原 #app / #start-screen 的 100vh 已在 style.css 内改为 100%,
   这里的高度即它们的实际高度。 */
.game1-stage {
```

```
  position: relative;
  height: calc(100vh - 132px);
  min-height: 620px;
  overflow: hidden;
  border-radius: 14px;
  box-shadow: 0 6px 24px rgba(16, 32, 64, .18);
}

/* ---------- 小屏兜底 ---------- */
@media (max-width: 1200px) {
  .game1-view { padding: 0 10px 14px; }
  .game1-stage { height: auto; min-height: 0; }
  .g1-title { font-size: 17px; }
  .g1-sub { display: none; }
}
</style>
/* ==================== 文件：src/game1/context.js ==================== */
/* ============================================================
   context.js —— 游戏一运行期上下文（ES Module 版新增模块）
   ------------------------------------------------------------
   职责（对应移植需求）：
     1. 保存当前挂载的 root 容器 / 游戏实例 / 上报回调 / 学生档案，
        替代原版依赖 window 的隐式全局（window.game 等）；
     2. 提供 root 作用域内的 DOM 查询助手 $ / $$，
        所有查询都限定在组件容器内，不会误伤站点其它元素；
     3. 统一登记 setTimeout / setInterval / requestAnimationFrame，
        destroy 时一次性清除，保证反复 mount / unmount 不残留定时器；
     4. 统一事件上报入口 emit()，payload 内的数字统一保留两位小数，
        并做一次深拷贝，确保上报出去的都是纯 JSON 可序列化对象。
   ============================================================ */

/* 全局唯一的运行期上下文（同一时刻只允许一局挂载） */
export const ctx = {
  root: null,          // HTMLElement：Game1View 的容器
  game: null,          // 游戏状态机实例（engine.js 中的 game）
  UI: null,            // UI 对象（main.js 注册）
  Transmission: null,  // 传导演示动画对象（main.js 注册）
  profile: null,       // 学生档案（{ displayName } 等）
  onEvent: null,       // 事件上报回调 (type, payload) => void
  alive: false,        // 是否处于挂载期；false 时所有异步回调自动失效
};

/* 异步任务登记表：destroy 时统一清理 */
const timerIds = new Set();
const frameIds = new Set();

/* ---------- DOM 查询助手（全部限定在 root 内） ---------- */

/** 在 root 内查询单个元素；未挂载或找不到时返回 null */
export function $(selector) {
```

```
  return ctx.root ? ctx.root.querySelector(selector) : null;
}

/** 在 root 内查询多个元素，返回真数组（便于 forEach / map） */
export function $$(selector) {
  return ctx.root ? Array.from(ctx.root.querySelectorAll(selector)) : [];
}

/* ---------- 异步任务助手 ---------- */

/**
 * 受控 setTimeout：挂载期结束后不再执行，并登记以便 destroy 清理
 * @returns {number} 定时器 id
 */
export function setTimer(fn, ms) {
  const id = setTimeout(() => {
    timerIds.delete(id);
    if (ctx.alive) fn();
  }, ms);
  timerIds.add(id);
  return id;
}

/**
 * 受控 setInterval：原版未使用，保留统一出口，避免将来出现"野定时器"
 */
export function setIntervalSafe(fn, ms) {
  const id = setInterval(() => {
    if (!ctx.alive) { clearInterval(id); timerIds.delete(id); return; }
    fn();
  }, ms);
  timerIds.add(id);
  return id;
}

/** 清除单个定时器（timeout / interval 在浏览器中共用 id 空间） */
export function clearTimer(id) {
  if (id === null || id === undefined) return;
  clearTimeout(id);
  clearInterval(id);
  timerIds.delete(id);
}

/**
 * 受控 requestAnimationFrame：挂载期结束后不再执行，并登记以便 destroy 取消
 * @returns {number} 帧 id
 */
export function requestFrame(fn) {
  const id = requestAnimationFrame((t) => {
    frameIds.delete(id);
```

```
    if (ctx.alive) fn(t);
  });
  frameIds.add(id);
  return id;
}

/** 清除所有定时器与动画帧（destroy 时调用） */
export function clearAllTimers() {
  timerIds.forEach((id) => {
    clearTimeout(id);
    clearInterval(id);
  });
  timerIds.clear();
  frameIds.forEach((id) => cancelAnimationFrame(id));
  frameIds.clear();
}

/* ---------- 事件上报 ---------- */

/** 数值统一保留两位小数，避免浮点噪声（如 0.30000000000000004） */
export function round2(v) {
  return Math.round(v * 100) / 100;
}

/**
 * 深拷贝并把所有数字四舍五入到两位小数
 * 用于保证上报 payload 是纯 JSON 对象（不含 DOM / 函数 / 循环引用）
 */
export function toPlain(value) {
  if (typeof value === 'number') return Number.isFinite(value) ? round2(value) : null;
  if (typeof value === 'function' || typeof value === 'undefined') return undefined;
  if (value === null || typeof value !== 'object') return value;
  if (Array.isArray(value)) return value.map(toPlain);
  const out = {};
  Object.keys(value).forEach((k) => {
    const v = toPlain(value[k]);
    if (v !== undefined) out[k] = v;
  });
  return out;
}

/**
 * 事件上报统一入口
 * @param {string} type    事件类型（game1.start / game1.policy / ...）
 * @param {object} payload 纯 JSON 可序列化的载荷（数字自动保留两位小数）
 */
export function emit(type, payload) {
  if (typeof ctx.onEvent !== 'function') return;
  try {
    ctx.onEvent(type, toPlain(payload || {}));
```

```
  } catch (e) {
    /* 上报失败不能影响游戏本身 */
    console.warn('[game1] onEvent 回调异常：', e);
  }
}
/* ==================== 文件：src/game1/data.js ==================== */
/* ============================================================
   data.js —— 游戏静态数据
   内容：政策卡牌（44 张）/ 事件脚本 / 因素链条 / 成就 / KPI 定义
   说明：经济数值单位为"指数点"（Y*=100 为潜在产出），
         展示层再换算为万亿元（×1.25，对齐中国现实体量）。
   ============================================================
   【ES Module 移植】顶层数据全部以 export 暴露，供 engine / main 等模块 import；
   原版靠 <script> 顺序 + 全局作用域共享，移植后不再有隐式全局。
   ============================================================ */

/* ---------- 政策卡牌 ----------
   changes: 对外生参数的调整（正负号即方向）
            G 政府购买 / T 税收 / M 货币供给 / NX0 自主净出口
            a 自主消费 / I0 自主投资 / Ystar 潜在产出
            nu 供给冲击(暂时) / e 汇率 / s SRAS 斜率(负=供给侧改善)
   temp: 临时政策，dur 个季度内每季重复生效
   quiz: 触发的题目类型（对应 quiz.js 中的题库键） */
export const CARDS = [
  /* ===== 财政政策（12 张，教材第 13 章 财政政策与货币政策的配合） ===== */
  { id:'f01', cat:'fiscal', name:'扩大政府购买', icon:'🏗️', quiz:'multG',
    desc:'加大基建与公共服务采购力度',
    effect:'政府购买 G +5', lag:'当季生效', side:'赤字扩大，可能挤出私人投资',
    formula:'k_G = 1/(1−β)', chapter:'第 13 章 第 2 节',
    changes:{ G:+5 } },
  { id:'f02', cat:'fiscal', name:'紧缩政府购买', icon:'✂️', quiz:'multG',
    desc:'压减一般性支出与低效投资',
    effect:'政府购买 G −4', lag:'当季生效', side:'短期压低总需求与就业',
    formula:'ΔY = k_G · ΔG', chapter:'第 13 章 第 2 节',
    changes:{ G:-4 } },
  { id:'f03', cat:'fiscal', name:'减税降费', icon:'📄', quiz:'multT',
    desc:'下调增值税与企业所得税负担',
    effect:'税收 T −5，企业投资 I0 +1', lag:'1 个季度', side:'财政收入下降',
    formula:'k_T = −β/(1−β)', chapter:'第 13 章 第 2 节',
    changes:{ T:-5, I0:+1 } },
  { id:'f04', cat:'fiscal', name:'增税补缺', icon:'💰', quiz:'multT',
    desc:'临时提高特定税目弥补赤字',
    effect:'税收 T +4', lag:'当季生效', side:'抑制消费与投资',
    formula:'ΔY = k_T · ΔT', chapter:'第 13 章 第 2 节',
    changes:{ T:+4 } },
  { id:'f05', cat:'fiscal', name:'发放消费券', icon:'🎟️', quiz:'multT',
    desc:'向居民定向发放消费补贴',
    effect:'自主消费 a +2（持续 2 季）', lag:'当季生效', side:'临时刺激，存在透支效应',
    formula:'C = a + β(Y−T)', chapter:'第 13 章 第 1 节',
    changes:{ a:+2 }, temp:true, dur:2 },
```

```
{ id:'f06', cat:'fiscal', name:'增发国债', icon:'📜', quiz:'crowdOut',
  desc:'发行长期建设国债为支出融资',
  effect:'政府购买 G +6，债务↑↑', lag:'当季生效', side:'利率上行，挤出效应明显',
  formula:'IS-LM：ΔG→r↑→I↓', chapter:'第 13 章 第 3 节',
  changes:{ G:+6 } },
{ id:'f07', cat:'fiscal', name:'提高个税起征点', icon:'👤', quiz:'multT',
  desc:'减轻中低收入群体税负',
  effect:'税收 T −3，公众信任 +3', lag:'当季生效', side:'财政空间收窄',
  formula:'Yd = Y − T', chapter:'第 13 章 第 1 节',
  changes:{ T:-3 }, trust:+3 },
{ id:'f08', cat:'fiscal', name:'民生服务采购', icon:'🏥', quiz:'gdpAccount',
  desc:'扩大养老、医疗等公共服务购买',
  effect:'政府购买 G +3，公众信任 +4', lag:'当季生效', side:'支出刚性较强',
  formula:'G 直接计入总需求', chapter:'第 12 章 第 1 节',
  changes:{ G:+3 }, trust:+4 },
{ id:'f09', cat:'fiscal', name:'企业设备补贴', icon:'🏭', quiz:'crowdOut',
  desc:'对制造业设备更新给予购置补贴',
  effect:'自主投资 I0 +2.5', lag:'1 个季度', side:'财政成本上升',
  formula:'I = I0 − d·r', chapter:'第 14 章 第 1 节',
  changes:{ I0:+2.5 } },
{ id:'f10', cat:'fiscal', name:'财政整顿', icon:'📊', quiz:'balanced',
  desc:'同时压支出、增收入以控债务',
  effect:'G −3 且 T +3', lag:'当季生效', side:'短期需求收缩',
  formula:'平衡预算乘数 = 1', chapter:'第 13 章 第 2 节',
  changes:{ G:-3, T:+3 } },
{ id:'f11', cat:'fiscal', name:'专项债提速', icon:'🚄', quiz:'multG',
  desc:'加快地方政府专项债发行使用',
  effect:'政府购买 G +4，债务↑', lag:'当季生效', side:'项目收益率下降',
  formula:'Y = C+I+G+NX', chapter:'第 13 章 第 2 节',
  changes:{ G:+4 } },
{ id:'f12', cat:'fiscal', name:'转移支付加码', icon:'🤝', quiz:'multT',
  desc:'增加对居民部门的转移支付',
  effect:'有效税收 T −4（等效）', lag:'1 个季度', side:'依赖财政可持续性',
  formula:'k_TR = β/(1−β)', chapter:'第 13 章 第 1 节',
  changes:{ T:-4 } },

/* ===== 货币政策（14 张，教材第 14、16 章） ===== */
{ id:'m01', cat:'monetary', name:'全面降准', icon:'🏦', quiz:'rr',
  desc:'下调法定存款准备金率 0.5pct',
  effect:'货币供给 M +5', lag:'1 个季度', side:'通胀与汇率压力',
  formula:'m = 1/rrd', chapter:'第 16 章 第 2 节',
  changes:{ M:+5 } },
{ id:'m02', cat:'monetary', name:'全面加准', icon:'🔒', quiz:'rr',
  desc:'上调准备金率冻结流动性',
  effect:'货币供给 M −5', lag:'1 个季度', side:'融资成本上升',
  formula:'ΔM = Δrrd 效应', chapter:'第 16 章 第 2 节',
  changes:{ M:-5 } },
{ id:'m03', cat:'monetary', name:'下调再贴现率', icon:'📋', quiz:'openMarket',
  desc:'央行降低对银行贴现融资成本',
  effect:'货币供给 M +4', lag:'当季生效', side:'信号意义大于数量',
```

```
    formula:'贴现窗口→基础货币↑', chapter:'第 16 章　第 2 节',
    changes:{ M:+4 } },
  { id:'m04', cat:'monetary', name:'上调再贴现率', icon:'🚫', quiz:'openMarket',
    desc:'收紧对银行的再贴现支持',
    effect:'货币供给 M −4', lag:'当季生效', side:'短期利率波动',
    formula:'贴现窗口→基础货币↓', chapter:'第 16 章　第 2 节',
    changes:{ M:-4 } },
  { id:'m05', cat:'monetary', name:'公开市场买入', icon:'🛒', quiz:'openMarket',
    desc:'央行从二级市场买入国债',
    effect:'货币供给 M +4', lag:'当季生效', side:'债券价格上涨',
    formula:'OMO 买入→基础货币↑', chapter:'第 16 章　第 2 节',
    changes:{ M:+4 } },
  { id:'m06', cat:'monetary', name:'公开市场卖出', icon:'📤', quiz:'openMarket',
    desc:'央行卖出债券回笼流动性',
    effect:'货币供给 M −4', lag:'当季生效', side:'利率上行',
    formula:'OMO 卖出→基础货币↓', chapter:'第 16 章　第 2 节',
    changes:{ M:-4 } },
  { id:'m07', cat:'monetary', name:'正回购', icon:'⏮️', quiz:'openMarket',
    desc:'短期卖出债券并约定回购',
    effect:'货币供给 M −3（持续 1 季）', lag:'当季生效', side:'仅为短期调节',
    formula:'正回购→暂时回笼', chapter:'第 16 章　第 2 节',
    changes:{ M:-3 }, temp:true, dur:1 },
  { id:'m08', cat:'monetary', name:'逆回购', icon:'⏭️', quiz:'openMarket',
    desc:'短期买入债券投放流动性',
    effect:'货币供给 M +3（持续 1 季）', lag:'当季生效', side:'跨季资金面波动',
    formula:'逆回购→暂时投放', chapter:'第 16 章　第 2 节',
    changes:{ M:+3 }, temp:true, dur:1 },
  { id:'m09', cat:'monetary', name:'MLF 超额续作', icon:'💧', quiz:'openMarket',
    desc:'中期借贷便利加量投放',
    effect:'货币供给 M +5', lag:'1 个季度', side:'银行负债成本变化',
    formula:'MLF→M2 派生↑', chapter:'第 16 章　第 3 节',
    changes:{ M:+5 } },
  { id:'m10', cat:'monetary', name:'MLF 缩量', icon:'🌵', quiz:'openMarket',
    desc:'中期流动性净回笼',
    effect:'货币供给 M −4', lag:'1 个季度', side:'市场利率上行',
    formula:'MLF→M2 派生↓', chapter:'第 16 章　第 3 节',
    changes:{ M:-4 } },
  { id:'m11', cat:'monetary', name:'TMLF 定向支持', icon:'🎯', quiz:'rr',
    desc:'定向支持小微与制造业的较长期资金',
    effect:'M +3，自主投资 I0 +1.5', lag:'2 个季度', side:'结构效应强于总量',
    formula:'定向降息→I↑', chapter:'第 16 章　第 3 节',
    changes:{ M:+3, I0:+1.5 } },
  { id:'m12', cat:'monetary', name:'SLF 应急投放', icon:'🚒', quiz:'openMarket',
    desc:'常备借贷便利满足短期流动性',
    effect:'M +2（持续 1 季）', lag:'当季生效', side:'利率走廊上限',
    formula:'SLF→隔夜流动性', chapter:'第 16 章　第 3 节',
    changes:{ M:+2 }, temp:true, dur:1 },
  { id:'m13', cat:'monetary', name:'差别准备金率', icon:'🧲', quiz:'rr',
    desc:'对达标机构实施更低准备金要求',
    effect:'M +3，结构性宽松', lag:'1 个季度', side:'激励相容设计复杂',
```

```
  formula:'m = 1/rrd（差异化）', chapter:'第 16 章　第 2 节',
  changes:{ M:+3 } },
{ id:'m14', cat:'monetary', name:'LPR 下调引导', icon:'📉', quiz:'monetaryChain',
  desc:'引导贷款市场报价利率下行',
  effect:'自主投资 I0 +2，公众信任 +2', lag:'1 个季度', side:'银行息差收窄',
  formula:'r↓ → I↑ → Y↑', chapter:'第 16 章　第 4 节',
  changes:{ I0:+2 }, trust:+2 },

/* ===== 产业政策（10 张，中国特供，供给侧视角） ===== */
{ id:'i01', cat:'industry', name:'供给侧结构性改革', icon:'🔧', quiz:'supplyShock',
  desc:'"三去一降一补"优化供给体系',
  effect:'潜在产出 Y* +2，SRAS 右移', lag:'3 个季度', side:'短期调整阵痛',
  formula:'Y*↑ → LRAS 右移', chapter:'第 17 章　第 3 节',
  changes:{ Ystar:+2, s:-0.1 } },
{ id:'i02', cat:'industry', name:'新基建计划', icon:'🗼', quiz:'multG',
  desc:'5G、数据中心等新型基础设施投资',
  effect:'G +3，潜在产出 Y* +1.5', lag:'2 个季度', side:'当期债务增加',
  formula:'G↑（短期），Y*↑（长期）', chapter:'第 17 章　第 2 节',
  changes:{ G:+3, Ystar:+1.5 } },
{ id:'i03', cat:'industry', name:'高新技术企业优惠', icon:'🔬', quiz:'multT',
  desc:'研发费用加计扣除等税收优惠',
  effect:'T −3，Y* +1.5', lag:'2 个季度', side:'财政减收',
  formula:'税收激励→技术进步', chapter:'第 17 章　第 2 节',
  changes:{ T:-3, Ystar:+1.5 } },
{ id:'i04', cat:'industry', name:'定向支持小微', icon:'🏪', quiz:'rr',
  desc:'普惠金融定向降准与信贷支持',
  effect:'I0 +1.5，M +2', lag:'1 个季度', side:'银行风险上升',
  formula:'信贷可得性→I↑', chapter:'第 17 章　第 2 节',
  changes:{ I0:+1.5, M:+2 } },
{ id:'i05', cat:'industry', name:'去产能攻坚', icon:'⛏️', quiz:'supplyShock',
  desc:'化解钢铁煤炭等行业过剩产能',
  effect:'SRAS 左移（P↑），随后 Y* +1', lag:'2 个季度', side:'短期供给收缩',
  formula:'s↑ → P↑；Y*↑ 后续', chapter:'第 17 章　第 3 节',
  changes:{ s:+0.15, Ystar:+1 } },
{ id:'i06', cat:'industry', name:'房住不炒调控', icon:'🏠', quiz:'isShift',
  desc:'抑制投机性住房需求',
  effect:'自主投资 I0 −2，金融稳定↑', lag:'当季生效', side:'相关产业链承压',
  formula:'I↓ → IS 左移', chapter:'第 17 章　第 1 节',
  changes:{ I0:-2 }, trust:+3 },
{ id:'i07', cat:'industry', name:'乡村振兴行动', icon:'🌾', quiz:'gdpAccount',
  desc:'加大农村基础设施与消费支持',
  effect:'a +1.5，G +1', lag:'2 个季度', side:'长期结构改善',
  formula:'C↑ + G↑', chapter:'第 17 章　第 2 节',
  changes:{ a:+1.5, G:+1 } },
{ id:'i08', cat:'industry', name:'双循环战略', icon:'🔄', quiz:'netExport',
  desc:'以国内大循环为主体、双循环互促',
  effect:'自主消费 a +1，NX0 +1', lag:'2 个季度', side:'见效较慢',
  formula:'内需↑ + 外需稳定', chapter:'第 17 章　第 2 节',
  changes:{ a:+1, NX0:+1 } },
{ id:'i09', cat:'industry', name:'环保限产', icon:'🌱', quiz:'supplyShock',
```

```
  desc:'重污染行业错峰生产',
  effect:'供给冲击 v +1.5，信任 +2', lag:'当季生效', side:'物价上行压力',
  formula:'v↑ → 菲利普斯曲线上移', chapter:'第 15 章 第 2 节',
  changes:{ nu:+1.5 }, trust:+2 },
{ id:'i10', cat:'industry', name:'统一大市场建设', icon:'🏞️', quiz:'supplyShock',
  desc:'破除地方保护、畅通要素流动',
  effect:'Y* +2，s −0.05', lag:'3 个季度', side:'改革协调成本高',
  formula:'交易成本↓ → Y*↑', chapter:'第 17 章 第 3 节',
  changes:{ Ystar:+2, s:-0.05 } },

/* ===== 贸易政策（8 张，教材第 18 章 开放经济） ===== */
{ id:'t01', cat:'trade', name:'提高关税', icon:'🛃', quiz:'netExport',
  desc:'对部分进口商品加征关税',
  effect:'净出口 NX0 +2，福利损失', lag:'当季生效', side:'报复风险、消费品涨价',
  formula:'NX = X − M(关税↓)', chapter:'第 18 章 第 1 节',
  changes:{ NX0:+2 }, trust:-2 },
{ id:'t02', cat:'trade', name:'降低关税', icon:'🚢', quiz:'netExport',
  desc:'扩大进口、降低平均关税水平',
  effect:'NX0 −2，消费物价回落', lag:'当季生效', side:'本土产业承压',
  formula:'进口↑ → NX↓', chapter:'第 18 章 第 1 节',
  changes:{ NX0:-2 }, trust:+2 },
{ id:'t03', cat:'trade', name:'汇率贬值干预', icon:'💱', quiz:'devalue',
  desc:'引导人民币适度贬值促进出口',
  effect:'NX0 +2.5，输入型通胀 v +0.5', lag:'1 个季度', side:'资本外流压力',
  formula:'e↑ → NX↑（马歇尔-勒纳）', chapter:'第 18 章 第 2 节',
  changes:{ NX0:+2.5, nu:+0.5, e:+0.15 } },
{ id:'t04', cat:'trade', name:'汇率升值干预', icon:'🏦', quiz:'devalue',
  desc:'引导升值降低进口成本',
  effect:'NX0 −2，v −0.5', lag:'1 个季度', side:'出口竞争力下降',
  formula:'e↓ → NX↓，P↓', chapter:'第 18 章 第 2 节',
  changes:{ NX0:-2, nu:-0.5, e:-0.15 } },
{ id:'t05', cat:'trade', name:'出口退税加码', icon:'📦', quiz:'netExport',
  desc:'提高出口商品退税率',
  effect:'NX0 +1.5', lag:'1 个季度', side:'财政补贴外需',
  formula:'X↑ → NX↑', chapter:'第 18 章 第 1 节',
  changes:{ NX0:+1.5 } },
{ id:'t06', cat:'trade', name:'签署自贸协定', icon:'🤝', quiz:'netExport',
  desc:'与主要伙伴深化关税互惠',
  effect:'NX0 +1.5，信任 +3', lag:'2 个季度', side:'谈判周期长',
  formula:'贸易创造效应', chapter:'第 18 章 第 3 节',
  changes:{ NX0:+1.5 }, trust:+3 },
{ id:'t07', cat:'trade', name:'外汇储备干预', icon:'🏛️', quiz:'devalue',
  desc:'动用外储平滑汇率过度波动',
  effect:'汇率波动收窄（v −0.3）', lag:'当季生效', side:'消耗外储',
  formula:'外汇市场干预', chapter:'第 18 章 第 2 节',
  changes:{ nu:-0.3 } },
{ id:'t08', cat:'trade', name:'资本流动管理', icon:'🚧', quiz:'isShift',
  desc:'加强跨境资本流动宏观审慎管理',
  effect:'减少热钱冲击（信任 +2）', lag:'当季生效', side:'开放度受限',
  formula:'资本管制→波动↓', chapter:'第 18 章 第 4 节',
```

```
    changes:{}, trust:+2 },
];

/* ---------- 外生变量中文名（传导演示 / 事件卡共用） ---------- */
export const EFFECT_NAMES = {
  a: '自主消费', I0: '自主投资', G: '政府购买', T: '税收',
  M: '货币供给', NX0: '净出口', Ystar: '潜在产出', s: '供给曲线斜率',
  nu: '供给冲击 v', e: '汇率',
};

/* ---------- 分类目录（用于抽屉展示） ---------- */
export const CATS = [
  { key:'fiscal',    name:'财政政策', icon:'🏛️' },
  { key:'monetary',  name:'货币政策', icon:'🏦' },
  { key:'industry',  name:'产业政策', icon:'⚙️' },
  { key:'trade',     name:'贸易政策', icon:'🚢' },
];

/* ---------- 事件脚本（前 8 季固定 + 随机池） ----------
   effects: 作用机制与政策卡一致，下一季结算时生效 */
export const SCRIPTED_EVENTS = [
  { q:1, icon:'🧧', title:'春节消费旺季', tag:'demand',
    desc:'居民集中消费，社零数据大幅走强，服务贸易景气度回升。',
    effects:{ a:+1.5 }, firm:['商圈客流创新高，餐饮零售企业加班备货。'] },
  { q:2, icon:'🦅', title:'美联储加息', tag:'supply',
    desc:'美联储宣布加息 25bp，中美利差收窄，资本流出压力上升。',
    effects:{ I0:-1, NX0:-0.5 }, firm:['外资企业表示融资成本上升，暂缓扩产计划。'] },
  { q:3, icon:'🛢️', title:'国际油价大涨', tag:'supply',
    desc:'地缘冲突推升原油价格，运输与化工成本普遍上行。',
    effects:{ nu:+2 }, firm:['物流企业上调运价，制造业成本压力增大。'] },
  { q:4, icon:'📢', title:'中央定调"稳增长"', tag:'policy',
    desc:'会议明确加大逆周期调节力度，市场信心明显改善。',
    effects:{ a:+1 }, firm:['企业纷纷表示政策预期更稳定，投资意愿回暖。'] },
  { q:5, icon:'⚔️', title:'贸易摩擦升级', tag:'supply',
    desc:'外部对华关税措施加码，出口订单出现波动。',
    effects:{ NX0:-2 }, firm:['外贸企业反映订单向东南亚分流，寻求转型。'] },
  { q:6, icon:'🤖', title:'人工智能产业突破', tag:'demand',
    desc:'国产大模型取得突破，数字基建与算力投资火热。',
    effects:{ Ystar:+1.5, I0:+1 }, firm:['科技企业大幅扩招算法工程师，算力中心满负荷运转。'] },
  { q:7, icon:'🏚️', title:'房地产下行压力', tag:'demand',
    desc:'部分房企债务风险暴露，地产链需求走弱。',
    effects:{ I0:-1.5 }, firm:['建材企业开工率下降，建筑机械租赁需求转弱。'] },
  { q:8, icon:'🌎', title:'全球复苏 外需回暖', tag:'demand',
    desc:'海外补库存周期启动，出口集装箱运价回升。',
    effects:{ NX0:+1.5 }, firm:['港口吞吐量回升，外贸工厂重启闲置产线。'] },
];
export const RANDOM_EVENTS = [
  { icon:'🌀', title:'台风灾害', tag:'supply',
    desc:'强台风登陆东南沿海，灾后重建需求上升。',
    effects:{ G:+1, nu:+0.5 }, firm:['受灾地区企业停工检修，保险理赔量激增。'] },
```

```
  { icon:'🔥', title:'通胀预期脱锚', tag:'supply',
    desc:'物价连续上涨引发居民抢购，通胀预期快速上行。',
    effects:{ nu:+1.5 }, firm:['超市出现囤货现象，企业提前囤积原材料。'] },
  { icon:'🏦', title:'银行间流动性紧张', tag:'policy',
    desc:'季末资金面骤紧，货币市场利率跳升。',
    effects:{ M:-3 }, firm:['中小企业反映票据贴现难度上升。'] },
  { icon:'😊', title:'消费信心回暖', tag:'demand',
    desc:'就业与收入预期改善，居民储蓄倾向下降。',
    effects:{ a:+1.5 }, firm:['文旅餐饮预订火爆，影院票房超预期。'] },
  { icon:'⚡', title:'电力供应紧张', tag:'supply',
    desc:'极端高温叠加用电高峰，部分地区有序用电。',
    effects:{ nu:+1 }, firm:['高耗能企业错峰生产，交货周期拉长。'] },
  { icon:'📉', title:'股市大幅调整', tag:'mixed',
    desc:'风险偏好回落，财富效应减弱。',
    effects:{ a:-1, I0:-0.5 }, firm:['投行裁员消息增多，企业 IPO 计划推迟。'] },
];

/* ---------- 因素链条（点击节点弹公式） ---------- */
export const CHAINS = {
  fiscal: [
    { t:'财政政策出台', know:{ f:'Y = C(Y−T) + I(r) + G + NX', x:'政府购买 G 与税收 T 是总需求的直接组成部分。扩张性财政政策通过增加 G 或减少 T 抬高总需求。', c:'第 13 章 第 1 节' } },
    { t:'IS 曲线右移', know:{ f:'r(Y) = [a − βT + I0 + G + NX − (1−β)Y] / d', x:'任一利率水平下的均衡产出都变大了，整条 IS 曲线向右移动，移动幅度 = 乘数 × 政策力度。', c:'第 14 章 第 1 节' } },
    { t:'总需求 AD↑', know:{ f:'AD：Y(P) 随 P↓ 而单调递增', x:'财政扩张使每一价格水平下的均衡产出上升，AD 曲线右移。', c:'第 15 章 第 1 节' } },
    { t:'产出 Y↑', know:{ f:'ΔY = k_G · ΔG，  k_G = 1/(1−β)', x:'乘数效应：支出→收入→消费→再支出……最终产出增量是初始支出的 1/(1−β) 倍。', c:'第 13 章 第 2 节' } },
    { t:'失业率 u↓', know:{ f:'奥肯定律：u − uN = −λ(Y−Y*)/Y*', x:'产出缺口转正后，企业扩大雇佣，失业率降至自然率之下。', c:'第 15 章 第 3 节' } },
    { t:'物价 P↑（副作用）', know:{ f:'菲利普斯：π = πe − β(u−uN) + v', x:'需求扩张以一定的通胀为代价；若经济已达充分就业，扩张将主要推高物价。', c:'第 15 章 第 2 节' } },
  ],
  monetary: [
    { t:'货币政策出台', know:{ f:'基础货币 B → 货币乘数 m = 1/rrd', x:'央行通过降准、公开市场操作等工具改变基础货币与货币乘数，从而调节货币供给 M。', c:'第 16 章 第 1 节' } },
    { t:'LM 曲线右移', know:{ f:'r(Y) = (k·Y − M/P)/h', x:'M 增加使任一产出水平对应的均衡利率下降，LM 曲线向右下方移动。', c:'第 14 章 第 2 节' } },
    { t:'利率 r↓', know:{ f:'货币市场：M/P = k·Y − h·r', x:'流动性供给增加，货币市场重新均衡时利率更低。', c:'第 14 章 第 2 节' } },
    { t:'投资 I↑', know:{ f:'I = I0 − d·r', x:'融资成本下降刺激企业固定资产投资与居民购房。', c:'第 14 章 第 1 节' } },
    { t:'产出 Y↑', know:{ f:'IS：Y = C + I(r) + G + NX', x:'投资是总需求的组成部分，I 上升通过乘数放大为产出增加。', c:'第 14 章 第 3 节' } },
    { t:'物价 P↑（副作用）', know:{ f:'MV = PY（数量方程）', x:'货币扩张长期中性：M 持续高增长最终主要体现为物价上涨。', c:'第 16 章 第 5 节' } },
  ],
  trade: [
    { t:'贸易政策出台', know:{ f:'NX = X(e) − M(e, Yd)', x:'关税与汇率通过进出口渠道影响净出口。
```

```
    ', c:'第 18 章 第 1 节' } },
    { t:'净出口 NX 变动', know:{ f:'马歇尔-勒纳条件：|εx|+|εm| > 1', x:'贬值改善贸易收支的前提是
    进出口需求弹性之和大于 1；短期还可能先恶化（J 曲线）。', c:'第 18 章 第 2 节' } },
    { t:'IS 曲线移动', know:{ f:'Y = C + I + G + NX(e, Y)', x:'NX 是四部门总需求的一部分，开放经
    济 IS 曲线更平坦。', c:'第 18 章 第 2 节' } },
    { t:'产出 Y 变动', know:{ f:'ΔY = k · ΔNX', x:'净出口变动同样经乘数放大作用于总产出。', c:'
    第 18 章 第 2 节' } },
    { t:'国际收支变化', know:{ f:'BP：经常账户 + 资本账户 = 0', x:'贸易顺差改善经常账户，但需与
    资本流动、汇率制度统筹考虑。', c:'第 18 章 第 3 节' } },
    { t:'物价/福利效应', know:{ f:'P = Pd + e·Pm（含进口投入）', x:'贬值推升进口价格形成输入型通
    胀；关税保护则带来消费者福利净损失。', c:'第 18 章 第 4 节' } },
  ],
};

/* ---------- 成就徽章 ---------- */
export const ACHIEVEMENTS = [
  { id:'growth',   name:'增长奇迹',       icon:'🚀', desc:'任一季度产出缺口 ≥ +3%',        cond:
    s => s.gap >= 3 },
  { id:'inflkill', name:'通胀终结者',     icon:'🛡️', desc:'连续 4 季通胀 ≤ 2.5%',          con
    d: s => s.piLowStreak >= 4 },
  { id:'fulljob',  name:'充分就业守护者', icon:'👷', desc:'连续 4 季失业率 ∈ [4%,6%]',     cond:
    s => s.uGoodStreak >= 4 },
  { id:'trap',     name:'流动性陷阱突破者', icon:'🕳️', desc:'利率 <1% 时仍实现增长转正',    co
    nd: s => s.trapWin },
  { id:'quiz10',   name:'题海学霸',       icon:'📚', desc:'累计答对 10 道政策题',          cond:
    s => s.quizRight >= 10 },
  { id:'debt',     name:'财政稳健奖',     icon:'⚖️', desc:'债务率始终 ≤ 65%',              cond:
    s => s.debtRatio <= 65 },
  { id:'steady',   name:'稳健舵手',       icon:'🎯', desc:'四维评分全部 ≥ 75',             cond:
    s => Object.values(s.radar).every(v => v >= 75) },
];

/* ---------- KPI 指标定义（顶部仪表盘） ---------- */
export const KPI_DEFS = [
  { key:'gdp',  name:'GDP',      unit:'万亿', fmt: v => (v*1.25).toFixed(1) },
  { key:'pi',   name:'通胀率',   unit:'%',    fmt: v => v.toFixed(1) },
  { key:'u',    name:'失业率',   unit:'%',    fmt: v => v.toFixed(1) },
  { key:'debt', name:'债务率',   unit:'%',    fmt: v => v.toFixed(0) },
  { key:'nx',   name:'净出口',   unit:'万亿', fmt: v => (v*1.25).toFixed(1) },
  { key:'trust',name:'信任度',   unit:'',     fmt: v => v.toFixed(0) },
];

/* ---------- 企业动态模板（按状态生成） ---------- */
export const FIRM_TPL = {
  hot:      ['订单排到下半年，工厂三班倒扩产能。', '招工难再现，一线工人工资上调。'],
  cold:     ['库存积压，企业降价去库存。', '部分产线停产检修，用工需求收缩。'],
  hiRate:   ['融资成本高企，制造业推迟设备投资。', '房企融资渠道收紧，拿地趋于谨慎。'],
  loRate:   ['贷款利率下行，企业信贷需求回暖。', '债券融资成本降低，龙头企业发债扩产。'],
  hiInfl:   ['原材料涨价挤压利润，企业提价转嫁成本。', '上游涨价传导至消费品，零售商调整标价。']
    ,
```

```
  defl:     ['工业品价格持续回落，企业盈利承压。', 'PPI 负增长区间，制造业投资回报率下降。'],
  exportUp: ['外贸订单回暖，集装箱一舱难求。', '出口企业加快交付节奏，港口忙碌。'],
  exportDn: ['外需走弱，出口企业转向内销。', '加工贸易企业开工率下降。'],
};
/* ==================== 文件：src/game1/models.js ==================== */
/* ============================================================
   models.js —— 宏观经济模型核心计算
   模型体系：IS-LM → AD-AS → 菲利普斯曲线/奥肯定律
   所有"产出"均为指数（潜在产出 Y* = 100），价格 P 为价格指数。
   【ES Module 移植】模型计算函数与常量全部 export，供 charts / engine / main import。
   ============================================================ */

export const MODEL = {
  /* —— 行为参数（教学简化，保持乘数关系与教材一致） ——
     校准目标：基线 Y≈100、r≈3%、π≈2%；IS-LM 支出乘数≈1.6（含挤出）；
     AD 斜率适中（实际余额效应显著），使经济在冲击后可自我收敛 */
  b: 0.8,      // β 边际消费倾向
  a0: 10,      // 自主消费基线
  I00: 11.75,  // 自主投资基线
  d: 3,        // 投资的利率敏感度
  k: 0.55,     // 货币需求的收入敏感度
  h: 4,        // 货币需求的利率敏感度
  lam: 0.45,   // 奥肯定律系数 λ
  kappa: 0.35, // 菲利普斯曲线产出缺口系数（与 SRAS 斜率 s 对应）
  piTarget: 2, // 通胀目标（预期锚）

  /* —— 初始外生状态 —— */
  init: {
    G: 18, T: 17,        // 政府购买 / 税收（指数点）
    M: 45,               // 货币供给（基期价格=100 下的名义货币）
    NX0: 2,              // 自主净出口
    Ystar: 100,          // 潜在产出
    P: 100,              // 价格指数
    s: 0.35,             // SRAS 斜率：P = Pe + s(Y−Y*)
    e: 7.1,              // 人民币汇率（展示用）
    nu: 0,               // 供给冲击（暂时项）
    piE: 2.0,            // 通胀预期 %
    uN: 5.0,             // 自然失业率 %
    debt: 60,            // 政府债务（指数点，展示×1.25=75 万亿）
    trust: 60,           // 公众信任度
  },
};

/* 构建某一季度结算用的外生参数包 ex
   供给冲击 ν 体现为 SRAS 截距上移（成本推动，滞胀效应） */
export function buildEx(st) {
  return {
    a: st.a, T: st.T, G: st.G, M: st.M, NX0: st.NX0,
    I0: st.I0, Ystar: st.Ystar, s: st.s, e: st.e,
    Pe: st.P * (1 + st.piE / 100) + (st.nu || 0),   // 预期价格 + 供给冲击
```

```
  };
}

/* —— AD：给定价格 P，由 IS-LM 联立求均衡产出 ——
   实际余额 = M × P0 / P（P0 = 100 为基期价格，M 为基期价格下的名义货币）
   Y(P) = [a − βT + I0 + G + NX0 + (d/h)·(M·P0/P)] / [(1−β) + d·k/h] */
export const P0 = 100;
export function realBalance(M, P) { return M * P0 / P; }

export function adY(P, ex) {
  const m = MODEL;
  const num = ex.a - m.b * ex.T + ex.I0 + ex.G + ex.NX0 + (m.d / m.h) * realBalance(ex.M, P);
  return num / ((1 - m.b) + m.d * m.k / m.h);
}

/* —— 给定 P 求该价格下的 IS-LM 均衡（Y, r） —— */
export function eqISLM(P, ex) {
  const Y = adY(P, ex);
  const r = (MODEL.k * Y - realBalance(ex.M, P)) / MODEL.h;
  return { Y: Y, r: r };
}

/* —— SRAS 反函数：P = Pe + s(Y − Y*) ⇒ Y = (P − Pe)/s + Y* —— */
export function asY(P, ex) {
  return (P - ex.Pe) / ex.s + ex.Ystar;
}

/* —— AD-AS 均衡：解 adY(P) = asY(P)（二分法，AD 单调减、AS 单调增，解唯一） —— */
export function equilibrate(ex) {
  let lo = 40, hi = 300;
  for (let i = 0; i < 60; i++) {
    const mid = (lo + hi) / 2;
    if (adY(mid, ex) - asY(mid, ex) > 0) lo = mid; else hi = mid;
  }
  const P = (lo + hi) / 2;
  const Y = adY(P, ex);
  const r = (MODEL.k * Y - realBalance(ex.M, P)) / MODEL.h;
  return { P: P, Y: Y, r: r };
}

/* —— 菲利普斯曲线：π = πe + κ·(Y−Y*)/Y* + ν —— */
export function phillipsPi(gapPct, piE, nu) {
  return piE + MODEL.kappa * gapPct + nu;
}

/* —— 奥肯定律：u = uN − λ·(Y−Y*)/Y* —— */
export function okunU(gapPct, uN) {
  return uN - MODEL.lam * gapPct;
}
```

```
/* ── 支出法分解：C + I + G + NX = Y（利用 IS 恒等式与 LM 利率） ── */
export function decompose(Y, r, ex) {
  const m = MODEL;
  const C = ex.a + m.b * (Y - ex.T);
  const I = Math.max(0.1, ex.I0 - m.d * r);
  const NX = ex.NX0;
  /* G 吸收残差，保证恒等成立（教学展示用） */
  const G = Y - C - I - NX;
  return { C: C, I: I, G: G, NX: NX };
}

/* ============================================================
   即时静态求解：外生参数变化后立即重解均衡（不改动状态）
   政策落地 / 事件冲击时调用，用于传导演示与实时反馈
   ============================================================ */
export function solveStatic(st) {
  const ex = buildEx(st);
  const eq = equilibrate(ex);
  const gap = (eq.Y - ex.Ystar) / ex.Ystar * 100;
  const pi = 100 * (eq.P / st.P - 1);
  const u = Math.max(0.5, okunU(gap, st.uN));
  const dec = decompose(eq.Y, eq.r, ex);
  return { Y: eq.Y, r: eq.r, P: eq.P, pi: pi, u: u, gap: gap, dec: dec };
}

/* ============================================================
   季度推进：货币适应增长 → 重解基准均衡 → 预期调整 → 债务核算
   （暂时供给冲击 v 在此消退，只影响其所在季度的均衡）
   ============================================================ */
export function advanceQuarter(st) {
  /* 0. 央行随通胀预期自动投放货币（匹配货币需求增长，
        否则实际余额侵蚀会让经济无政策时陷入衰退漂移） */
  st.M = st.M * (1 + st.piE / 100);

  /* 1. 新季度基准均衡 */
  const solved = solveStatic(st);

  /* 2. 通胀预期：适应性预期 + 向 2% 目标锚定
        （0.45 惯性 / 0.4 学习 / 0.15 央行信誉锚） */
  const piE = Math.max(-2, 0.45 * st.piE + 0.4 * solved.pi + 0.15 * MODEL.piTarget);

  /* 3. 政府债务：赤字累积（盈余时可部分偿还） */
  let debt = st.debt;
  if (st.G > st.T) debt += (st.G - st.T) * 0.5;
  else debt -= (st.T - st.G) * 0.25;

  Object.assign(st, solved, { piE: piE, debt: debt, nu: 0 });
  return solved;
}
```

```
/** 明天的日期串 YYYY-MM-DD（错题重置后明天开始复习） */
function tomorrow() {
  const d = new Date()
  d.setDate(d.getDate() + 1)
  return d.toISOString().slice(0, 10)
}

/* 错题列表 */
router.get('/list', async (req, res, next) => {
  try {
    const userId = resolveUserId(req)
    const rows = await query(
      `SELECT question_id, snapshot, my_answer, reason, times, ease_factor, interval_days, repet
    itions, due_date, last_review_at
       FROM wrong_book WHERE user_id = ? ORDER BY created_at DESC`,
      [userId]
    )
    const list = rows.map((r) => ({
      questionId: r.question_id,
      snapshot: r.snapshot,
      myAnswer: r.my_answer,
      reason: r.reason,
      times: r.times,
      easeFactor: Number(r.ease_factor),
      intervalDays: r.interval_days,
      repetitions: r.repetitions,
      dueDate: r.due_date,
      lastReviewAt: r.last_review_at
    }))
    res.json({ success: true, code: 'OK', message: '', data: { list } })
  } catch (e) { next(e) }
})

/* 收入错题（UPSERT：已存在则次数+1、重置 SM-2 计划） */
router.post('/add', async (req, res, next) => {
  try {
    const userId = resolveUserId(req)
    const { questionId, snapshot, myAnswer, reason } = req.body || {}
    if (questionId === undefined || !snapshot) {
      return res.status(400).json({ success: false, code: 'PARAM_ERROR', message: '缺少 question
    Id 或题目快照', data: null })
    }

    await query(
      `INSERT INTO wrong_book (user_id, question_id, snapshot, my_answer, reason, times, due_dat
    e)
       VALUES (?, ?, ?, ?, ?, 1, ?)
       ON DUPLICATE KEY UPDATE
         times = times + 1,
         my_answer = VALUES(my_answer),
```

```
         reason = COALESCE(VALUES(reason), reason),
         ease_factor = 2.50, interval_days = 0, repetitions = 0,
         due_date = VALUES(due_date), last_review_at = NULL`,
      [
        userId, Number(questionId),
        JSON.stringify(snapshot),
        myAnswer !== undefined && myAnswer !== null ? JSON.stringify(myAnswer) : null,
        reason || null,
        tomorrow()
      ]
    )
    res.json({ success: true, code: 'OK', message: '', data: { added: true } })
  } catch (e) { next(e) }
})

/* 移除错题 */
router.post('/remove', async (req, res, next) => {
  try {
    const userId = resolveUserId(req)
    const { questionId } = req.body || {}
    if (questionId === undefined) {
      return res.status(400).json({ success: false, code: 'PARAM_ERROR', message: '缺少 question
    Id', data: null })
    }
    await query('DELETE FROM wrong_book WHERE user_id = ? AND question_id = ?', [userId, Number(
    questionId)])
    res.json({ success: true, code: 'OK', message: '', data: { removed: true } })
  } catch (e) { next(e) }
})

/* 标注错误原因 */
router.post('/reason', async (req, res, next) => {
  try {
    const userId = resolveUserId(req)
    const { questionId, reason } = req.body || {}
    if (questionId === undefined) {
      return res.status(400).json({ success: false, code: 'PARAM_ERROR', message: '缺少 question
    Id', data: null })
    }
    await query('UPDATE wrong_book SET reason = ? WHERE user_id = ? AND question_id = ?', [reaso
    n || null, userId, Number(questionId)])
    res.json({ success: true, code: 'OK', message: '', data: { updated: true } })
  } catch (e) { next(e) }
})

export default router
/* ==================== 文件：server/routes/review.js ==================== */
/**
 * 错题复习路由 /api/review（SM-2 间隔重复）
 * ------------------------------------------------
```

```
 * GET  /api/review/due   今天到期的复习队列（按到期先后，上限 20）
 * POST /api/review/grade 复习评分：remember/fuzzy/forgot → SM-2 调度下次复习
 */
import { Router } from 'express'
import { query } from '../db.js'
import { resolveUserId } from '../middleware/auth.js'
import { sm2Schedule, GRADE_MAP } from '../services/reviewService.js'

const router = Router()

/* 到期复习队列 */
router.get('/due', async (req, res, next) => {
  try {
    const userId = resolveUserId(req)
    const rows = await query(
      `SELECT question_id, snapshot, my_answer, reason, times, ease_factor, interval_days, repet
    itions, due_date
       FROM wrong_book
       WHERE user_id = ? AND due_date <= CURDATE()
       ORDER BY due_date ASC
       LIMIT 20`,
      [userId]
    )
    const list = rows.map((r) => ({
      questionId: r.question_id,
      snapshot: r.snapshot,
      myAnswer: r.my_answer,
      reason: r.reason,
      times: r.times,
      dueDate: r.due_date
    }))
    res.json({ success: true, code: 'OK', message: '', data: { list } })
  } catch (e) { next(e) }
})

/* 复习评分：三档按钮 → SM-2 计算下次到期日 */
router.post('/grade', async (req, res, next) => {
  try {
    const userId = resolveUserId(req)
    const { questionId, grade } = req.body || {}
    if (questionId === undefined || !GRADE_MAP[grade]) {
      return res.status(400).json({ success: false, code: 'PARAM_ERROR', message: '缺少 question
    Id 或 grade 非法（remember/fuzzy/forgot）', data: null })
    }

    /* 取当前调度状态 */
    const rows = await query(
      'SELECT ease_factor, interval_days, repetitions FROM wrong_book WHERE user_id = ? AND ques
    tion_id = ?',
      [userId, Number(questionId)]
```

```
    )
    if (rows.length === 0) {
      return res.status(404).json({ success: false, code: 'NOT_FOUND', message: '错题不存在', da
    ta: null })
    }
    const row = rows[0]

    /* SM-2 计算新调度 */
    const next = sm2Schedule(
      { easeFactor: Number(row.ease_factor), intervalDays: row.interval_days, repetitions: row.r
    epetitions },
      GRADE_MAP[grade]
    )

    await query(
      `UPDATE wrong_book
       SET ease_factor = ?, interval_days = ?, repetitions = ?, due_date = ?, last_review_at = N
    OW()
       WHERE user_id = ? AND question_id = ?`,
      [next.easeFactor, next.intervalDays, next.repetitions, next.dueDate, userId, Number(questi
    onId)]
    )

    res.json({
      success: true, code: 'OK', message: '',
      data: { nextDueDate: next.dueDate, intervalDays: next.intervalDays, easeFactor: next.easeF
    actor }
    })
  } catch (e) { next(e) }
})

export default router
/* ==================== 文件：server/routes/islm.js ==================== */
/**
 * IS-LM 后端计算路由 /api/islm
 * ------------------------------------------------
 * POST /api/islm/simulate  单情景求解（曲线点 + 均衡点 + 解读文本）
 * POST /api/islm/compare   基准/冲击双情景对比
 * 补齐前端 src/api/islm.js 注释里预留的"正式接口"，
 * 前端调用失败时仍会回落本地计算，保证离线可玩。
 */
import { Router } from 'express'
import { calcIslmCurves, generateAnalysisNote } from '../services/islmSolver.js'

const router = Router()

/** 必填九参数校验（非负 + 数字化） */
function sanitizeParams(raw) {
  const KEYS = ['C0', 'c', 'I0', 'd', 'G', 'T', 'MP', 'k', 'h']
  const p = {}
```

```
  for (const k of KEYS) {
    const v = Number(raw?.[k])
    if (raw?.[k] === undefined || raw?.[k] === null || Number.isNaN(v) || v < 0) return null
    p[k] = v
  }
  return p
}

/* 单情景求解 */
router.post('/simulate', (req, res) => {
  const p = sanitizeParams(req.body)
  if (!p) {
    return res.status(400).json({ success: false, code: 'PARAM_ERROR', message: '参数缺失或非法
    （九个参数必须为非负数字）', data: null })
  }
  const result = calcIslmCurves(p)
  if (!result.valid) {
    return res.status(400).json({ success: false, code: 'PARAM_ERROR', message: result.reason, d
    ata: null })
  }
  const note = generateAnalysisNote(p, result)
  res.json({ success: true, code: 'OK', message: '', data: { ...result, note } })
})

/* 基准/冲击对比 */
router.post('/compare', (req, res) => {
  const base = sanitizeParams(req.body?.base)
  const shock = sanitizeParams(req.body?.shock)
  if (!base || !shock) {
    return res.status(400).json({ success: false, code: 'PARAM_ERROR', message: 'base/shock 参数
    缺失或非法', data: null })
  }
  const baseResult = calcIslmCurves(base)
  const shockResult = calcIslmCurves(shock)
  if (!baseResult.valid) {
    return res.status(400).json({ success: false, code: 'PARAM_ERROR', message: '基准参数无效：'
     + baseResult.reason, data: null })
  }
  if (!shockResult.valid) {
    return res.status(400).json({ success: false, code: 'PARAM_ERROR', message: '冲击参数无效：'
     + shockResult.reason, data: null })
  }
  const note = generateAnalysisNote(base, baseResult, shockResult)
  res.json({ success: true, code: 'OK', message: '', data: { base: baseResult, shock: shockResul
    t, note } })
})

export default router
/* ==================== 文件：server/routes/agent.js ==================== */
/**
```

```
 * AI 智能体路由 /api/agent（SSE 流式）
 * ------------------------------------------------
 * POST /api/agent/chat
 * 入参：{ userId, messages: [{role, content}] }
 * 事件流（text/event-stream，每条事件一行 `data: {json}\n\n`）：
 *   { event:'tool_call',   {name, args} }   模型请求调用工具
 *   { event:'tool_result', {name, summary} } 工具执行完成
 *   { event:'delta',       {text} }         回答文本增量（打字机）
 *   { event:'reset',       {} }             丢弃当前半截文本（工具轮开始前）
 *   { event:'done',        {model, toolCalls} } 回答结束
 *   { event:'error',       {message} }      异常
 * 另每 15s 发一行 `: ping` 心跳，防止代理空闲断连。
 */
import { Router } from 'express'
import { runAgent } from '../services/agentService.js'
import { resolveUserId } from '../middleware/auth.js'

const router = Router()

/** SSE 基础响应头 */
function sseHeaders(res) {
  res.status(200)
  res.set({
    'Content-Type': 'text/event-stream; charset=utf-8',
    'Cache-Control': 'no-cache',
    Connection: 'keep-alive',
    'X-Accel-Buffering': 'no' // 告诉 nginx 类代理不要缓冲
  })
  res.flushHeaders()
}

/** 发送一条 SSE 事件 */
function sseSend(res, event, payload) {
  res.write(`event: ${event}\ndata: ${JSON.stringify(payload)}\n\n`)
}

router.post('/chat', async (req, res) => {
  sseHeaders(res)

  const { messages } = req.body || {}
  if (!Array.isArray(messages) || messages.length === 0) {
    sseSend(res, 'error', { message: 'messages 不能为空' })
    return res.end()
  }

  const userId = resolveUserId(req)

  /* 心跳定时器：SSE 空闲期发注释行保活 */
  const heartbeat = setInterval(() => res.write(': ping\n\n'), 15000)
```

```
  try {
    /* runAgent 边执行边 emit，这里转成 SSE 事件 */
    const result = await runAgent({
      userId,
      messages: messages.slice(-12), // 最多带最近 12 条上下文，控制 token
      emit: (event, payload) => sseSend(res, event, payload)
    })
    sseSend(res, 'done', { model: result.model, toolCalls: result.toolCalls })
  } catch (e) {
    console.error('[agent] 对话异常：\n', e.stack || e)
    sseSend(res, 'error', { message: '智能体暂时不可用，请稍后再试' })
  } finally {
    clearInterval(heartbeat)
    res.end()
  }
})

export default router
/* ==================== 文件：server/routes/game.js ==================== */
/**
 * 多人博弈游戏 · REST 辅助路由（对局报告查询）
 * ------------------------------------------------
 * WebSocket 负责实时对局，这里只做事后查询：
 *  - GET /api/game/reports?userId=xxx   我的历史对局列表（画像"游戏表现"数据源）
 *  - GET /api/game/report/:code         单局详情（决策序列 + 聊天 + 雷达图）
 */
import { Router } from 'express'
import { query } from '../db.js'

const router = Router()

/* 我的历史对局（按时间倒序，默认最近 20 局） */
router.get('/reports', async (req, res, next) => {
  try {
    const userId = req.query.userId
    if (!userId) {
      return res.json({ success: false, code: 'PARAM_ERROR', message: '缺少 userId', data: null
    })
    }
    /* 一个玩家可能在一局中留下多条决策，用 DISTINCT 去重取对局 */
    const rows = await query(
      `SELECT DISTINCT r.id, r.room_code, r.total_rounds, r.final_metrics, r.report, r.created_a
    t
       FROM game_room r
       JOIN game_decision d ON d.room_id = r.id
       WHERE d.user_id = ?
       ORDER BY r.created_at DESC
       LIMIT 20`,
      [userId]
    )
```

```
    res.json({
      success: true, code: 'OK', message: '',
      data: rows.map(r => ({
        id: r.id,
        code: r.room_code,
        rounds: r.total_rounds,
        metrics: r.final_metrics,
        verdict: r.report?.verdict || null,
        radar: r.report?.radar || null,
        createdAt: r.created_at
      }))
    })
  } catch (e) {
    next(e)
  }
})

/* 单局详情（复盘用：决策序列 + 聊天 + 完整报告） */
router.get('/report/:code', async (req, res, next) => {
  try {
    const [room] = await query(
      'SELECT id, room_code, total_rounds, final_metrics, report, created_at FROM game_room WHER
    E room_code = ?',
      [req.params.code]
    )
    if (!room) {
      return res.json({ success: false, code: 'NOT_FOUND', message: '对局不存在', data: null })
    }
    const decisions = await query(
      'SELECT round_no, role, nickname, decision, metrics_after FROM game_decision WHERE room_id
     = ? ORDER BY round_no, FIELD(role, "government", "bank", "enterprise", "consumer")',
      [room.id]
    )
    const chat = await query(
      'SELECT nickname, role, text, created_at FROM game_chat WHERE room_id = ? ORDER BY id',
      [room.id]
    )
    res.json({
      success: true, code: 'OK', message: '',
      data: { room, decisions, chat }
    })
  } catch (e) {
    next(e)
  }
})

export default router
/* ==================== 文件：server/routes/game1.js ==================== */
/**
 * 游戏一（经济舵手·单机政策模拟）路由 /api/game1
```

```
 * ================================================================
 * 该游戏的引擎完全在前端运行（14 个季度、44 张政策卡、六步传导动画），
 * 后端只负责两件事：
 *   1. 保存终局快照（game1_session）——供画像的"游戏决策力"维度使用
 *   2. 提供历史查询——学生可回看自己的政策路线与评分变化
 *
 * 过程中的细粒度操作（每张政策、每次答题）走 /api/event 事件流，
 * 避免把高频操作写成数据库大事务。
 */
import { Router } from 'express'
import { query, safeQuery } from '../db.js'
import { resolveUserId } from '../middleware/auth.js'

const router = Router()

/** 数值裁剪（防止前端异常数据把统计拉飞） */
function clampNum(v, min, max, fallback = 0) {
  const n = Number(v)
  if (!Number.isFinite(n)) return fallback
  return Math.max(min, Math.min(max, n))
}

/**
 * POST /api/game1/session
 * body: {
 *   mode:'learn'|'sandbox', quarters, totalScore, radar, kpiSeries,
 *   achievements, policyCount, quizCorrect, quizTotal, nickname, startedAt
 * }
 */
router.post('/session', async (req, res, next) => {
  try {
    const userId = resolveUserId(req)
    const b = req.body || {}
    const mode = b.mode === 'sandbox' ? 'sandbox' : 'learn'

    const result = await query(
      `INSERT INTO game1_session
         (user_id, nickname, mode, quarters, total_score, radar, kpi_series,
          achievements, policy_count, quiz_correct, quiz_total, started_at, finished_at)
       VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, NOW())`,
      [
        userId,
        b.nickname ? String(b.nickname).slice(0, 50) : null,
        mode,
        clampNum(b.quarters, 0, 60, 14),
        b.totalScore === undefined || b.totalScore === null ? null : clampNum(b.totalScore, 0, 1
    00, 0),
        b.radar ? JSON.stringify(b.radar) : null,
        b.kpiSeries ? JSON.stringify(Array.isArray(b.kpiSeries) ? b.kpiSeries.slice(0, 60) : [])
     : null,
```

```
        b.achievements ? JSON.stringify(Array.isArray(b.achievements) ? b.achievements.slice(0,
    30) : []) : null,
        clampNum(b.policyCount, 0, 999, 0),
        clampNum(b.quizCorrect, 0, 999, 0),
        clampNum(b.quizTotal, 0, 999, 0),
        b.startedAt ? new Date(b.startedAt) : null
      ]
    )

    res.json({ success: true, code: 'OK', message: '', data: { id: result.insertId } })
  } catch (e) { next(e) }
})

/**
 * GET /api/game1/sessions?limit=10
 * 当前学生的历史对局（最新在前）
 */
router.get('/sessions', async (req, res, next) => {
  try {
    const userId = resolveUserId(req)
    const limit = Math.min(Number(req.query.limit) || 10, 50)
    const rows = await safeQuery(
      `SELECT id, mode, quarters, total_score, radar, policy_count, quiz_correct, quiz_total,
              achievements, finished_at
         FROM game1_session WHERE user_id = ? ORDER BY id DESC LIMIT ?`,
      [userId, limit]
    )
    const asObj = (v) => {
      if (!v) return null
      if (typeof v === 'object') return v
      try { return JSON.parse(v) } catch { return null }
    }
    res.json({
      success: true,
      code: 'OK',
      message: '',
      data: {
        list: rows.map((r) => ({
          id: r.id,
          mode: r.mode,
          quarters: r.quarters,
          score: r.total_score === null ? null : Number(r.total_score),
          radar: asObj(r.radar),
          achievements: asObj(r.achievements) || [],
          policyCount: r.policy_count,
          quizCorrect: r.quiz_correct,
          quizTotal: r.quiz_total,
          finishedAt: r.finished_at
        }))
      }
```

```
    })
  } catch (e) { next(e) }
})

/**
 * GET /api/game1/stats
 * 当前学生的游戏一汇总（画像页/首页卡片用）
 */
router.get('/stats', async (req, res, next) => {
  try {
    const userId = resolveUserId(req)
    const rows = await safeQuery(
      `SELECT COUNT(*) AS sessions, MAX(total_score) AS best, AVG(total_score) AS avgScore,
              SUM(policy_count) AS policies, SUM(quiz_correct) AS quizCorrect,
              SUM(quiz_total) AS quizTotal, MAX(finished_at) AS lastAt
         FROM game1_session WHERE user_id = ?`,
      [userId]
    )
    const r = rows[0] || {}
    res.json({
      success: true,
      code: 'OK',
      message: '',
      data: {
        sessions: Number(r.sessions) || 0,
        bestScore: r.best === null || r.best === undefined ? null : Math.round(Number(r.best) *
    10) / 10,
        avgScore: r.avgScore === null || r.avgScore === undefined ? null : Math.round(Number(r.a
    vgScore) * 10) / 10,
        policies: Number(r.policies) || 0,
        quizCorrect: Number(r.quizCorrect) || 0,
        quizTotal: Number(r.quizTotal) || 0,
        lastAt: r.lastAt || null
      }
    })
  } catch (e) { next(e) }
})

export default router
/* ==================== 文件：server/routes/event.js ==================== */
/**
 * 学习行为事件采集路由 /api/event
 * ================================================================
 * 两游戏与学习页面的操作统一走这里落库（learning_event 表），
 * 是"AI 能读取学生在游戏中的操作"的数据入口。
 *
 * 设计要点：
 *   - 批量上报：前端按 1~2 秒攒批，减少请求数（游戏动画期间事件密集）
 *   - 白名单校验：source 只允许 game1/game2/learning/ai，避免脏数据污染画像
 *   - 负载裁剪：payload 序列化后超过 2000 字符直接截断，防止恶意/异常大包
```

```
 *   - 匿名可用：未登录学生也能上报（user_id 用 hintUserId），登录后自动归到 u_<id>
 */
import { Router } from 'express'
import { query } from '../db.js'
import { resolveUserId } from '../middleware/auth.js'

const router = Router()

/** 允许的事件来源 */
const SOURCES = new Set(['game1', 'game2', 'learning', 'ai'])
/** 单次上报最多事件数 */
const MAX_BATCH = 60
/** payload 序列化后的最大长度 */
const MAX_PAYLOAD = 2000

/** 校验并规整单条事件 */
function normalizeEvent(raw) {
  if (!raw || typeof raw !== 'object') return null
  const source = String(raw.source || '')
  if (!SOURCES.has(source)) return null
  const type = String(raw.type || '').slice(0, 40)
  if (!type) return null

  let payload = null
  if (raw.payload && typeof raw.payload === 'object') {
    try {
      const text = JSON.stringify(raw.payload)
      payload = text.length > MAX_PAYLOAD ? text.slice(0, MAX_PAYLOAD) : text
    } catch {
      payload = null
    }
  }
  const duration = Number(raw.durationMs)
  return [
    source,
    type,
    payload,
    Number.isFinite(duration) && duration >= 0 ? Math.min(duration, 24 * 3600 * 1000) : null
  ]
}

/**
 * POST /api/event/batch
 * body: { events: [{ source, type, payload?, durationMs? }], hintUserId? }
 */
router.post('/batch', async (req, res, next) => {
  try {
    const list = Array.isArray(req.body?.events) ? req.body.events : []
    if (list.length === 0) {
      return res.status(400).json({ success: false, code: 'PARAM_ERROR', message: 'events 不能为
```

```
    空', data: null })
    }
    const userId = resolveUserId(req)
    const rows = list.slice(0, MAX_BATCH).map(normalizeEvent).filter(Boolean)
    if (rows.length === 0) {
      return res.status(400).json({ success: false, code: 'PARAM_ERROR', message: '没有合法事件'
    , data: null })
    }

    /* 多值 INSERT：一次往返写入整批事件 */
    const placeholders = rows.map(() => '(?, ?, ?, ?, ?)').join(', ')
    const params = []
    for (const [source, type, payload, duration] of rows) {
      params.push(userId, source, type, payload, duration)
    }
    await query(
      `INSERT INTO learning_event (user_id, source, event_type, payload, duration_ms)
       VALUES ${placeholders}`,
      params
    )

    res.json({ success: true, code: 'OK', message: '', data: { accepted: rows.length } })
  } catch (e) { next(e) }
})

/**
 * POST /api/event/one
 * 单条上报（低频事件，如游戏结束时的一次性上报）
 */
router.post('/one', async (req, res, next) => {
  try {
    const row = normalizeEvent(req.body)
    if (!row) {
      return res.status(400).json({ success: false, code: 'PARAM_ERROR', message: '事件格式不正
    确', data: null })
    }
    const userId = resolveUserId(req)
    await query(
      `INSERT INTO learning_event (user_id, source, event_type, payload, duration_ms)
       VALUES (?, ?, ?, ?, ?)`,
      [userId, ...row]
    )
    res.json({ success: true, code: 'OK', message: '', data: { accepted: 1 } })
  } catch (e) { next(e) }
})

export default router
/* ==================== 文件：server/routes/knowledge.js ==================== */
/**
 * 知识图谱路由 /api/knowledge
```

```
 * ================================================================
 * 提供"知识点学习"部分的结构化数据：
 *   GET /api/knowledge/graph         整张图谱（节点 + 边 + 本人节点掌握度）
 *   GET /api/knowledge/stats         图谱规模指标
 *   GET /api/knowledge/nodes/:id     节点详情（教材原文块 + 关联题目 + 先修/后继）
 *   GET /api/knowledge/path          本人的推荐学习路径（学习前沿 + 拓扑序）
 *   GET /api/knowledge/search?q=     关键词检索节点
 *
 * 掌握度口径：node_mastery 表（BKT 节点级）；未作答节点返回 null，
 * 前端用浅灰显示"未学"，与"已学但薄弱"区分开。
 */
import { Router } from 'express'
import { resolveUserId } from '../middleware/auth.js'
import { getNodeMasteryMap } from '../services/nodeMasteryService.js'
import {
  knowledgeNodes, knowledgeEdges, nodeMap, graphForClient, graphStats,
  frontierNodes, topoOrder, TOPIC_TO_NODES
} from '../knowledge/graph.js'
import { allChunks } from '../knowledge/chunks.js'
import { questionBank } from '../../src/data/questionBank.js'

const router = Router()

/** chunkId → chunk（节点详情展示教材原文用） */
const chunkMap = new Map(allChunks.map((c) => [c.chunkId, c]))
/** questionId → 题目（节点详情展示关联练习用） */
const questionMap = new Map(questionBank.map((q) => [q.id, q]))

/** 题目脱敏：只保留题干与考点，去掉选项判定与答案数值 */
function safeQuestion(q) {
  return {
    id: q.id,
    topic: q.topic,
    type: q.type,
    difficulty: q.difficulty,
    question: q.question,
    examPoint: q.examPoint,
    relatedSection: q.relatedSection
  }
}

/** 图谱（含学生掌握度） */
router.get('/graph', async (req, res, next) => {
  try {
    const userId = resolveUserId(req)
    const mastery = await getNodeMasteryMap(userId)
    const graph = graphForClient(mastery)

    /* 附加：每个节点的先修/后继，前端点击节点时可高亮链路 */
    const children = new Map(knowledgeNodes.map((n) => [n.id, []]))
```

```
    for (const e of knowledgeEdges) {
      if (e.type === 'prereq') children.get(e.source).push(e.target)
    }
    for (const n of graph.nodes) {
      n.prereq = (nodeMap.get(n.id)?.prereq) || []
      n.children = children.get(n.id) || []
    }

    res.json({
      success: true,
      code: 'OK',
      message: '',
      data: { ...graph, stats: { ...graphStats(), chunks: allChunks.length, exercises: questionB
    ank.length }, topicsMap: Object.fromEntries(TOPIC_TO_NODES) }
    })
  } catch (e) { next(e) }
})

/** 图谱规模指标（含知识库块数，说明"知识库 + 知识图谱"两层结构） */
router.get('/stats', (_req, res) => {
  res.json({
    success: true,
    code: 'OK',
    message: '',
    data: {
      ...graphStats(),
      chunks: allChunks.length,
      exercises: questionBank.length,
      chunkTypes: [...new Set(allChunks.map((c) => c.type))]
    }
  })
})

/** 节点详情 */
router.get('/nodes/:id', async (req, res, next) => {
  try {
    const userId = resolveUserId(req)
    const node = nodeMap.get(req.params.id)
    if (!node) {
      return res.status(404).json({ success: false, code: 'NOT_FOUND', message: '知识节点不存在'
    , data: null })
    }
    const mastery = await getNodeMasteryMap(userId)

    /* 教材原文块（保留标题与正文，供"展开看教材"） */
    const chunks = node.chunkIds
      .map((cid) => chunkMap.get(cid))
      .filter(Boolean)
      .map((c) => ({ chunkId: c.chunkId, title: c.title, type: c.type, content: c.content.slice(
    0, 1200) }))
```

```
    /* 后继节点（学完这个下一步能学什么） */
    const children = knowledgeEdges
      .filter((e) => e.type === 'prereq' && e.source === node.id)
      .map((e) => nodeMap.get(e.target))
      .filter(Boolean)
      .map((n) => ({ id: n.id, name: n.name }))

    res.json({
      success: true,
      code: 'OK',
      message: '',
      data: {
        ...node,
        prereqNodes: node.prereq.map((id) => ({ id, name: nodeMap.get(id)?.name })).filter((x) =
    > x.name),
        children,
        chunks,
        questions: node.questionIds.map((qid) => questionMap.get(qid)).filter(Boolean).map(safeQ
    uestion),
        mastery: mastery.has(node.id) ? mastery.get(node.id) : null
      }
    })
  } catch (e) { next(e) }
})

/**
 * 推荐学习路径
 * 结合两件事：
 *   1. 图结构：拓扑序保证先修关系（不会推荐"还没学基础就去学极端情形"）
 *   2. 个人掌握度：前沿节点优先、掌握度低的排前面
 */
router.get('/path', async (req, res, next) => {
  try {
    const userId = resolveUserId(req)
    const mastery = await getNodeMasteryMap(userId)
    const frontier = frontierNodes(mastery, 0.6, 8)
    const order = topoOrder()

    const path = frontier.map((id, idx) => {
      const n = nodeMap.get(id)
      return {
        step: idx + 1,
        id: n.id,
        name: n.name,
        topic: n.topic,
        section: n.section,
        difficulty: n.difficulty,
        mastery: mastery.has(id) ? Math.round(mastery.get(id) * 1000) / 1000 : null,
        reason: mastery.has(id) ? '先修已具备，且当前掌握度最低' : '先修已具备，尚未开始学习'
```

```
      }
    })

    /* 全部节点的拓扑顺序（前端可画"学习地图"） */
    const fullPath = order.map((id, idx) => {
      const n = nodeMap.get(id)
      return {
        index: idx + 1,
        id,
        name: n.name,
        difficulty: n.difficulty,
        mastery: mastery.has(id) ? Math.round(mastery.get(id) * 1000) / 1000 : null
      }
    })

    res.json({ success: true, code: 'OK', message: '', data: { frontier: path, order: fullPath }
     })
  } catch (e) { next(e) }
})

/** 关键词检索节点（按名称/摘要/关键词匹配，返回相关度排序） */
router.get('/search', (req, res) => {
  const q = String(req.query.q || '').trim().toLowerCase()
  if (!q) {
    return res.json({ success: true, code: 'OK', message: '', data: { list: [] } })
  }
  const scored = knowledgeNodes.map((n) => {
    const fields = [n.name, n.summary, n.topic, ...n.keywords].map((s) => String(s).toLowerCase(
    ))
    let score = 0
    for (const f of fields) {
      if (f === q) score += 3
      else if (f.includes(q)) score += 2
      else if (q.includes(f) && f.length >= 2) score += 1
    }
    return { node: n, score }
  }).filter((x) => x.score > 0).sort((a, b) => b.score - a.score).slice(0, 10)

  res.json({
    success: true,
    code: 'OK',
    message: '',
    data: {
      list: scored.map((x) => ({
        id: x.node.id,
        name: x.node.name,
        topic: x.node.topic,
        section: x.node.section,
        summary: x.node.summary,
        score: x.score
```

```
      }))
    }
  })
})

export default router
/* ==================== 文件：server/routes/ai.js ==================== */
/**
 * AI 能力路由 /api/ai
 * ================================================================
 *   GET  /api/ai/profile      学生画像（带 30 分钟缓存，?force=1 强制重算）
 *   GET  /api/ai/advice       针对性学习建议（画像 + 强化学习策略）
 *   GET  /api/ai/recommend    单次动作推荐（供"下一步学什么"卡片）
 *   GET  /api/ai/model        DQN 模型与训练数据状态（前端"AI 引擎"卡片）
 *   POST /api/ai/train        用历史经验离线训练（管理/演示用，需登录）
 *
 * 说明：画像与建议的生成会调用大模型（无 Key 自动走确定性模板），
 * 因此这里用 30s 超时保护；单次请求内不会重复训练模型。
 */
import { Router } from 'express'
import { requireAuth, resolveUserId } from '../middleware/auth.js'
import { buildProfile } from '../services/profileService.js'
import { generateAdvice, getLatestAdvice } from '../services/adviceService.js'
import { recommend, modelStatus, trainFromHistory } from '../services/rlService.js'

const router = Router()

/** 学生画像 */
router.get('/profile', async (req, res, next) => {
  try {
    const userId = resolveUserId(req)
    const force = req.query.force === '1' || req.query.force === 'true'
    const profile = await buildProfile(userId, { force, useLLM: true })
    res.json({ success: true, code: 'OK', message: '', data: profile })
  } catch (e) { next(e) }
})

/** 学习建议（GET 时若已有建议可只读；?refresh=1 重新生成） */
router.get('/advice', async (req, res, next) => {
  try {
    const userId = resolveUserId(req)
    const refresh = req.query.refresh === '1' || req.query.refresh === 'true'
    if (!refresh) {
      const latest = await getLatestAdvice(userId)
      if (latest) {
        return res.json({ success: true, code: 'OK', message: '', data: latest })
      }
    }
    const advice = await generateAdvice(userId, { force: refresh, useLLM: true })
    res.json({ success: true, code: 'OK', message: '', data: advice })
```

```
  } catch (e) { next(e) }
})

/** 单次推荐（不重新生成建议，只返回策略动作） */
router.get('/recommend', async (req, res, next) => {
  try {
    const userId = resolveUserId(req)
    const data = await recommend(userId, { record: req.query.record !== '0' })
    res.json({ success: true, code: 'OK', message: '', data })
  } catch (e) { next(e) }
})

/** 模型状态 */
router.get('/model', async (_req, res, next) => {
  try {
    res.json({ success: true, code: 'OK', message: '', data: await modelStatus() })
  } catch (e) { next(e) }
})

/** 离线训练（登录后可触发，用于演示"训练 → 推荐变化"） */
router.post('/train', requireAuth, async (req, res, next) => {
  try {
    const steps = Math.max(10, Math.min(Number(req.body?.steps) || 200, 2000))
    const scope = req.body?.scope === 'all' ? null : resolveUserId(req)
    const result = await trainFromHistory(steps, scope)
    res.json({ success: true, code: 'OK', message: '', data: result })
  } catch (e) { next(e) }
})

export default router
/* ==================== 文件：server/routes/teacher.js ==================== */
/**
 * 教师端路由 /api/teacher（全部要求教师角色）
 * ================================================================
 *   GET  /api/teacher/overview            班级概览（首屏指标 + 知识点热力 + 活跃趋势）
 *   GET  /api/teacher/students            学生列表（含风险分级）
 *   GET  /api/teacher/student/:userId     单个学生详情（画像/建议/作答/游戏/提示）
 *   POST /api/teacher/student/:userId/advice  重新生成该生学习建议
 *   POST /api/teacher/insight             AI 班级学情简报
 *   GET  /api/teacher/export              导出学情 CSV
 *   GET  /api/teacher/classes             班级列表（下拉筛选用）
 *   GET  /api/teacher/activity            最近行为流（课堂实时看板）
 */
import { Router } from 'express'
import { query, safeQuery } from '../db.js'
import { requireTeacher } from '../middleware/teacher.js'
import {
  classOverview, listStudents, studentDetail, classInsight, exportStudentsCsv, parseUserId
} from '../services/teacherService.js'
import { generateAdvice } from '../services/adviceService.js'
```

```
const router = Router()

/* 该路由组内所有接口都要求教师身份 */
router.use(requireTeacher)

/** 班级概览 */
router.get('/overview', async (req, res, next) => {
  try {
    const className = req.query.className ? String(req.query.className) : null
    res.json({ success: true, code: 'OK', message: '', data: await classOverview({ className })
    })
  } catch (e) { next(e) }
})

/** 学生列表 */
router.get('/students', async (req, res, next) => {
  try {
    const className = req.query.className ? String(req.query.className) : null
    const limit = Math.min(Number(req.query.limit) || 200, 500)
    const list = await listStudents({ className, limit })
    res.json({ success: true, code: 'OK', message: '', data: { list, total: list.length } })
  } catch (e) { next(e) }
})

/** 学生详情 */
router.get('/student/:userId', async (req, res, next) => {
  try {
    const detail = await studentDetail(req.params.userId)
    res.json({ success: true, code: 'OK', message: '', data: detail })
  } catch (e) {
    if (e.message === '学生不存在' || e.message === '非法的学生标识') {
      return res.status(404).json({ success: false, code: 'NOT_FOUND', message: e.message, data:
     null })
    }
    next(e)
  }
})

/** 为该生重新生成学习建议（教师点"一键生成指导"） */
router.post('/student/:userId/advice', async (req, res, next) => {
  try {
    const uid = parseUserId(req.params.userId)
    if (!uid) {
      return res.status(400).json({ success: false, code: 'PARAM_ERROR', message: '非法的学生标
    识', data: null })
    }
    const advice = await generateAdvice(`u_${uid}`, { force: true, useLLM: true })
    res.json({ success: true, code: 'OK', message: '', data: advice })
  } catch (e) { next(e) }
```

```
})

/** 班级 AI 学情简报 */
router.post('/insight', async (req, res, next) => {
  try {
    const className = req.body?.className ? String(req.body.className) : null
    res.json({ success: true, code: 'OK', message: '', data: await classInsight({ className }) }
    )
  } catch (e) { next(e) }
})

/** 导出 CSV */
router.get('/export', async (req, res, next) => {
  try {
    const className = req.query.className ? String(req.query.className) : null
    const csv = await exportStudentsCsv({ className })
    const filename = encodeURIComponent(`学情导出_${className || '全部班级'}_${new Date().toISOS
    tring().slice(0, 10)}.csv`)
    res.setHeader('Content-Type', 'text/csv; charset=utf-8')
    res.setHeader('Content-Disposition', `attachment; filename*=UTF-8''${filename}`)
    res.send(csv)
  } catch (e) { next(e) }
})

/** 班级列表 */
router.get('/classes', async (_req, res, next) => {
  try {
    const rows = await safeQuery(
      `SELECT COALESCE(class_name, '默认班级') AS name, COUNT(*) AS students
         FROM users WHERE role = 'student' GROUP BY class_name ORDER BY students DESC`
    )
    res.json({ success: true, code: 'OK', message: '', data: { list: rows } })
  } catch (e) { next(e) }
})

/** 最近行为流（课堂看板：谁在玩什么、做到哪一步） */
router.get('/activity', async (req, res, next) => {
  try {
    const limit = Math.min(Number(req.query.limit) || 30, 100)
    const rows = await safeQuery(
      `SELECT e.user_id, u.display_name, e.source, e.event_type, e.payload, e.created_at
         FROM learning_event e
         LEFT JOIN users u ON CONCAT('u_', u.id) = e.user_id
        ORDER BY e.id DESC LIMIT ?`,
      [limit]
    )
    const list = rows.map((r) => ({
      userId: r.user_id,
      name: r.display_name || (r.user_id === 'anonymous' ? '匿名学生' : r.user_id),
      source: r.source,
```

```
      type: r.event_type,
      payload: typeof r.payload === 'string' ? safeJson(r.payload) : r.payload,
      at: r.created_at
    }))
    res.json({ success: true, code: 'OK', message: '', data: { list } })
  } catch (e) { next(e) }
})

/** 教师端首页所需的"待办"计数（重点学生 + 今日活跃） */
router.get('/summary', async (_req, res, next) => {
  try {
    const rows = await query(
      `SELECT COUNT(*) AS total FROM users WHERE role = 'student'`
    )
    const todayRows = await safeQuery(
      `SELECT COUNT(DISTINCT user_id) AS n FROM answer_record WHERE DATE(created_at) = CURDATE()
    `
    )
    res.json({
      success: true,
      code: 'OK',
      message: '',
      data: {
        students: Number(rows[0]?.total) || 0,
        activeToday: Number(todayRows[0]?.n) || 0
      }
    })
  } catch (e) { next(e) }
})

function safeJson(v) {
  try { return JSON.parse(v) } catch { return null }
}

export default router
/* ==================== 文件：server/prompts/profile.txt ==================== */
【角色】
你是"宏观经济学智能学习系统"的学生画像分析师。你面对的是一名学习《IS-LM 模型》的本科生，你的任务
    是根据系统采集到的客观学习数据，生成一份**准确、克制、可执行**的学生画像。

【输入】
User Prompt 会给你一份 JSON 统计，包含：
- knowledge：5 个知识点（IS 曲线 / LM 曲线 / IS-LM 均衡 / 政策运用 / 模型评析）的 BKT 掌握概率、作
    答次数、正确率
- nodes：25 个知识节点的掌握情况与薄弱节点
- habits：做题量、活跃天数、平均答题耗时、近 7 天表现
- hint：阶梯导学使用情况（各层级次数、是否直取完整解析、提示依赖度）
- game：游戏一（单机政策模拟）与游戏二（多人博弈）的操作表现
- radar：六维能力雷达（0~100）
```

【写作铁律】
1. 只依据输入数据说话，**禁止编造**未出现的数据、题号、分数。
2. 数据不足（例如完全没有作答记录）时，明确写"数据不足，暂按新同学处理"，不要臆测。
3. 不下"聪明/笨"这类价值判断，只描述**可观察的学习行为与知识状态**。
4. 语言面向学生本人，第二人称"你"，语气像助教面批：具体、鼓励、可执行。
5. 每一条不足都必须配一条可操作的动作（做什么题、看哪个节点、用哪种模式）。
6. 不泄露任何题目的最终答案数值。

【输出格式】
只输出合法 JSON，不要输出 JSON 以外的任何文字：
```
{
  "summary": "一句话画像（30 字以内，点出最重要的特征）",
  "narrative": "画像正文（200~320 字，分 2~3 段：知识状态 → 学习行为 → 游戏中的决策风格）",
  "tags": ["3~5 个标签，每个不超过 8 字"],
  "strengths": ["1~3 条优势，每条不超过 20 字"],
  "weaknesses": ["1~3 条短板，每条不超过 20 字"],
  "focus_nodes": ["最该补的 1~3 个知识节点名称"]
}
/* ==================== 文件：server/prompts/advice.txt ==================== */
```
【角色】
你是"宏观经济学智能学习系统"的学习规划师。你要把一份学生画像和一条由强化学习策略给出的"下一知识节点"建议，翻译成学生看得懂、做得到的学习计划。

【输入】
User Prompt 给你的 JSON 包含：
- profile：学生画像（雷达六维、薄弱知识点、薄弱节点、学习习惯、提示依赖度、游戏表现）
- policy：强化学习策略推荐的下一个知识节点（node_id / name / topic / 推荐理由 / Q 值）
- candidates：候选知识节点及其掌握度（策略在多个动作上的估值）

【写作铁律】
1. 建议必须**落在具体动作**上：做哪个知识点的哪类题、看哪个节点、玩哪个游戏模式练什么。
2. 优先级由数据决定：掌握度最低且处于学习前沿的节点排最前。
3. 每天的任务量要现实（单次 20~40 分钟），不要一次给 10 条。
4. 结合游戏数据给"迁移练习"建议（例如财政政策薄弱 → 去游戏一执行扩张性财政并观察挤出效应）。
5. 不编造数据，不提具体题目的答案数值。

【输出格式】
只输出合法 JSON：
```
{
  "headline": "本次学习计划的一句话主题（20 字以内）",
  "narrative": "计划说明（120~200 字，解释为什么先学这个、学完能解决什么问题）",
  "items": [
    {
      "priority": 1,
      "title": "动作标题（15 字以内）",
      "action": "具体做什么（40 字以内，含入口：知识点学习/题库练习/游戏一/游戏二）",
      "reason": "为什么做这个（引用数据，30 字以内）",
      "minutes": 25,
      "target_node": "知识节点 ID"
    }
```

```
  ]
}
items 给 3~5 条，按 priority 升序。
/* ==================== 文件：server/prompts/teacher.txt ==================== */
【角色】
你是"宏观经济学智能学习系统"的教学分析师，面向任课教师撰写班级学情简报。读者是老师，不是学生，因
    此要**结论先行、数据支撑、给出教学动作**。

【输入】
User Prompt 给出一份班级 JSON：学生人数、平均掌握度、正确率分布、活跃度、薄弱知识点分布、提示依
    赖度、游戏参与情况、需要重点关注的学生名单。

【写作铁律】
1. 先给结论（这个班整体处在什么水平、最突出的问题是什么），再给证据（具体百分比、人数）。
2. 不评价学生个人品性，只描述学习状态；点名以"待关注学生"形式给出，避免贴标签。
3. 给出 2~4 条**课堂上可执行**的教学动作（讲什么、练什么、用什么游戏活动）。
4. 只使用输入数据，不编造人数与比例。
5. 语言简洁，总字数控制在 400 字以内。

【输出格式】
只输出合法 JSON：
{
  "headline": "班级学情一句话结论（25 字以内）",
  "overview": "整体情况（120~180 字，含关键数据）",
  "problems": ["2~3 条主要问题，每条不超过 30 字"],
  "actions": ["2~4 条教学建议，每条不超过 40 字，要能落到课堂动作"],
  "focus_students": [{"name": "学生昵称", "reason": "关注理由（20 字以内）"}]
}
/* ==================== 文件：server/sql/init.sql ==================== */
-- ============================================================
-- 阶梯导学功能 · MySQL 初始化脚本
-- 使用方式：mysql -u root -p < server/sql/init.sql
-- 幂等：重复执行不会破坏已有数据（IF NOT EXISTS）
-- ============================================================

-- 建库（utf8mb4 支持全部中文与特殊符号）
CREATE DATABASE IF NOT EXISTS islm_platform
  DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE islm_platform;

-- ------------------------------------------------------------
-- 表 1：导学会话（每名学生 × 每道题 仅一条记录）
-- 保存该题的提示解锁进度与检索快照，是"每题独立保存提示层级"的载体
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS hint_session (
  id                BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id           VARCHAR(64)  NOT NULL COMMENT '学生标识（前端 localStorage 生成）',
  question_id       INT          NOT NULL COMMENT '题目编号 1~8',
  current_level     TINYINT      NOT NULL DEFAULT 0 COMMENT '已解锁层级 0 未解锁/1/2/3，单调递增
    不回退',
```

```
  solution_unlocked TINYINT(1)   NOT NULL DEFAULT 0 COMMENT '是否已查看 L4 完整解析',
  rag_snapshot      JSON         NULL COMMENT '首次 RAG 检索固化的知识块集合，L1~L3 复用保证口径一
    致',
  attempt_count     INT          NOT NULL DEFAULT 0 COMMENT '带作答信息的提示请求次数（卡壳次数
    口径）',
  confusion_text    VARCHAR(500) NULL COMMENT '学生最近一次填写的困惑描述',
  created_at        DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at        DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  UNIQUE KEY uk_user_question (user_id, question_id)
) ENGINE=InnoDB COMMENT='阶梯导学会话表';

-- ------------------------------------------------------------
-- 表 2：提示日志（每次模型输出一条）
-- 同时作为前端刷新后恢复历史提示卡片的数据源
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS hint_log (
  id                BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  session_id        BIGINT UNSIGNED NOT NULL COMMENT '关联 hint_session.id',
  user_id           VARCHAR(64)  NOT NULL,
  question_id       INT          NOT NULL,
  level             TINYINT      NOT NULL COMMENT '本次输出层级 1~4',
  trigger_type      VARCHAR(20)  NOT NULL DEFAULT 'HINT_STEP' COMMENT 'HINT_STEP 逐级/SOLUTION_DI
    RECT 直取 L4/CONFUSION_RETRY 困惑重试',
  content           MEDIUMTEXT   NOT NULL COMMENT '模型返回并通过校验后的 JSON 字符串',
  rag_chunk_ids     JSON         NULL COMMENT '本次引用的知识块 ID 数组',
  model_name        VARCHAR(100) NULL COMMENT '模型名（mock 模式记 mock）',
  prompt_tokens     INT          NULL,
  completion_tokens INT          NULL,
  degraded          TINYINT(1)   NOT NULL DEFAULT 0 COMMENT '是否为降级直出知识块原文',
  feedback          VARCHAR(8)   NULL COMMENT '学生对本条提示的评价 up/down',
  created_at        DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  KEY idx_session (session_id),
  KEY idx_user_question (user_id, question_id)
) ENGINE=InnoDB COMMENT='阶梯导学提示日志表';

-- ------------------------------------------------------------
-- 表 3：注册用户（JWT 登录体系的账号表）
-- 登录后错题本 / 学习记录可云端同步，未登录仍走浏览器本地存储
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS users (
  id            BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  username      VARCHAR(50)  NOT NULL COMMENT '登录用户名 3~20 字符',
  password_hash VARCHAR(100) NOT NULL COMMENT 'bcrypt 哈希（60 字符，留余量）',
  display_name  VARCHAR(50)  NULL COMMENT '昵称（默认同用户名）',
  created_at    DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uk_username (username)
) ENGINE=InnoDB COMMENT='注册用户表';

-- ------------------------------------------------------------
-- 表 4：作答记录（每次提交答案一条，BKT 知识追踪的数据来源）
```

```
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS answer_record (
  id             BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id        VARCHAR(64)  NOT NULL COMMENT '登录用户 u_数字 或匿名 hintUserId',
  question_id    INT          NOT NULL COMMENT '题目编号',
  topic          VARCHAR(50)  NOT NULL COMMENT '知识点（BKT 按此维度追踪）',
  q_type         VARCHAR(10)  NOT NULL COMMENT '题型 choice/blank/judge 等',
  difficulty     VARCHAR(10)  NOT NULL COMMENT '难度 easy/medium/hard',
  is_correct     TINYINT(1)   NOT NULL COMMENT '是否答对 1 对 0 错',
  student_answer JSON         NULL COMMENT '学生作答内容',
  duration_ms    INT          NULL COMMENT '本题作答耗时（毫秒）',
  created_at     DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  KEY idx_user_topic (user_id, topic),
  KEY idx_user_time (user_id, created_at)
) ENGINE=InnoDB COMMENT='作答记录表';

-- ------------------------------------------------------------
-- 表 5：知识点掌握状态（BKT 模型输出，每用户 × 每知识点一条）
-- p_known 为贝叶斯知识追踪估计的"当前已掌握概率"
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS mastery_state (
  id            BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id       VARCHAR(64)  NOT NULL,
  topic         VARCHAR(50)  NOT NULL,
  p_known       DECIMAL(5,4) NOT NULL DEFAULT 0.1500 COMMENT '掌握概率先验 0.15',
  attempts      INT          NOT NULL DEFAULT 0 COMMENT '累计作答次数',
  correct_count INT          NOT NULL DEFAULT 0 COMMENT '累计答对次数',
  updated_at    DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  UNIQUE KEY uk_user_topic (user_id, topic)
) ENGINE=InnoDB COMMENT='知识点掌握状态表（BKT）';

-- ------------------------------------------------------------
-- 表 6：云端错题本（登录用户多端同步，含 SM-2 间隔重复调度字段）
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS wrong_book (
  id            BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id       VARCHAR(64)  NOT NULL,
  question_id   INT          NOT NULL COMMENT '题目编号',
  snapshot      JSON         NOT NULL COMMENT '题目完整快照（防题库改动）',
  my_answer     JSON         NULL COMMENT '最近一次错误作答',
  reason        VARCHAR(100) NULL COMMENT '错误原因标注',
  times         INT          NOT NULL DEFAULT 1 COMMENT '累计做错次数',
  ease_factor   DECIMAL(4,2) NOT NULL DEFAULT 2.50 COMMENT 'SM-2 难度因子 EF',
  interval_days INT          NOT NULL DEFAULT 0 COMMENT 'SM-2 当前复习间隔（天）',
  repetitions   INT          NOT NULL DEFAULT 0 COMMENT 'SM-2 连续通过次数',
  due_date      DATE         NOT NULL COMMENT '下次复习到期日',
  last_review_at DATETIME    NULL COMMENT '最近一次复习评分时间',
  created_at    DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uk_user_question (user_id, question_id),
  KEY idx_user_due (user_id, due_date)
```

```
) ENGINE=InnoDB COMMENT='云端错题本（SM-2 调度）';

-- ------------------------------------------------------------
-- 表 7：知识块向量表（RAG 语义检索用，启动时为空会自动构建）
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS knowledge_embedding (
  chunk_id VARCHAR(64)  PRIMARY KEY COMMENT '知识块 ID，对应 chunks.js 的 id',
  model    VARCHAR(100) NOT NULL COMMENT '向量模型名或 pseudo（本地伪向量）',
  dim      INT          NOT NULL COMMENT '向量维度',
  vector   JSON         NOT NULL COMMENT '浮点数组（JSON 存储）',
  created_at DATETIME   NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB COMMENT='知识块向量表';

-- ------------------------------------------------------------
-- 表 8：智能体对话日志（答辩演示可回放完整对话与工具调用过程）
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS agent_log (
  id         BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id    VARCHAR(64)  NOT NULL,
  role       VARCHAR(16)  NOT NULL COMMENT 'user/assistant/tool',
  content    MEDIUMTEXT   NULL COMMENT '消息文本',
  tool_calls JSON         NULL COMMENT '工具调用明细数组',
  created_at DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  KEY idx_user_time (user_id, created_at)
) ENGINE=InnoDB COMMENT='智能体对话日志表';

-- ------------------------------------------------------------
-- 表 9：多人博弈对局主表（终局写入，含完整博弈报告 JSON）
-- 供"个性化画像"的游戏表现维度与教师复盘读取
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS game_room (
  id            BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  room_code     VARCHAR(8)   NOT NULL COMMENT '4 位房间码',
  host_user_id  VARCHAR(64)  NULL COMMENT '房主用户标识（政府位）',
  status        VARCHAR(20)  NOT NULL DEFAULT 'finished' COMMENT 'finished（只存终局）',
  total_rounds  INT          NOT NULL DEFAULT 3 COMMENT '总回合数',
  final_metrics JSON         NULL COMMENT '终局指标 {g,u,pi,r,Y,gap}',
  report        JSON         NULL COMMENT '完整博弈报告（事件/决策序列/雷达图/判定）',
  created_at    DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uk_code (room_code),
  KEY idx_host (host_user_id)
) ENGINE=InnoDB COMMENT='多人博弈对局表';

-- ------------------------------------------------------------
-- 表 10：博弈决策序列（每回合 × 每角色一条，决策风格分析数据源）
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS game_decision (
  id            BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  room_id       BIGINT UNSIGNED NOT NULL COMMENT '关联 game_room.id',
  round_no      INT          NOT NULL COMMENT '回合 1~4',
```

```
  role          VARCHAR(20)  NOT NULL COMMENT 'government/bank/enterprise/consumer',
  user_id       VARCHAR(64)  NULL COMMENT '真人玩家标识（AI 决策为 NULL）',
  nickname      VARCHAR(50)  NOT NULL,
  decision      JSON         NOT NULL COMMENT '决策增量 如 {"dG":6,"dT":0}',
  metrics_after JSON         NULL COMMENT '该回合结算后指标',
  created_at    DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  KEY idx_room_round (room_id, round_no),
  KEY idx_user (user_id)
) ENGINE=InnoDB COMMENT='博弈决策序列表';

-- ------------------------------------------------------------
-- 表 11：博弈协商聊天记录（小组博弈报告的对话内容）
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS game_chat (
  id         BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  room_id    BIGINT UNSIGNED NOT NULL COMMENT '关联 game_room.id',
  user_id    VARCHAR(64)  NULL COMMENT '真人玩家标识',
  nickname   VARCHAR(50)  NOT NULL,
  role       VARCHAR(20)  NOT NULL,
  text       VARCHAR(300) NOT NULL COMMENT '聊天内容（上限 300 字）',
  created_at DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  KEY idx_room (room_id)
) ENGINE=InnoDB COMMENT='博弈聊天记录表';

-- ============================================================
-- 以下为「知识图谱 + 学生画像 + 教师端 + 强化学习」扩展表
-- 设计原则：与既有表解耦，全部 CREATE TABLE IF NOT EXISTS，可重复执行
-- ============================================================

-- ------------------------------------------------------------
-- 表 12：知识节点掌握度（BKT 的节点级口径，图谱着色与 RL 状态来源）
-- 与 mastery_state 的区别：mastery_state 按 5 个知识点统计，本表按 25 个知识节点统计
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS node_mastery (
  id            BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id       VARCHAR(64)  NOT NULL,
  node_id       VARCHAR(40)  NOT NULL COMMENT '知识节点 ID，对应 knowledge/graph.js',
  topic         VARCHAR(50)  NOT NULL COMMENT '所属知识点，便于按主题聚合',
  p_known       DECIMAL(5,4) NOT NULL DEFAULT 0.1500 COMMENT '节点掌握概率',
  attempts      INT          NOT NULL DEFAULT 0,
  correct_count INT          NOT NULL DEFAULT 0,
  updated_at    DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  UNIQUE KEY uk_user_node (user_id, node_id),
  KEY idx_user_topic (user_id, topic)
) ENGINE=InnoDB COMMENT='知识节点掌握度表（BKT 节点级）';

-- ------------------------------------------------------------
-- 表 13：学习行为事件流（统一采集两游戏操作与学习行为，AI 画像的数据底座）
-- source 区分来源：game1 单机政策模拟 / game2 多人博弈 / learning 学习行为 / ai AI 交互
-- ------------------------------------------------------------
```

```
CREATE TABLE IF NOT EXISTS learning_event (
  id         BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id    VARCHAR(64)  NOT NULL,
  source     VARCHAR(16)  NOT NULL COMMENT 'game1/game2/learning/ai',
  event_type VARCHAR(40)  NOT NULL COMMENT '如 game1.policy、learning.answer',
  payload    JSON         NULL COMMENT '事件负载（已裁剪为轻量字段）',
  duration_ms INT         NULL COMMENT '本次行为耗时',
  created_at DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  KEY idx_user_time (user_id, created_at),
  KEY idx_user_source (user_id, source),
  KEY idx_type (event_type)
) ENGINE=InnoDB COMMENT='学习行为事件流表';

-- ------------------------------------------------------------
-- 表 14：游戏一对局记录（单机政策模拟的终局快照）
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS game1_session (
  id           BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id      VARCHAR(64)  NOT NULL,
  nickname     VARCHAR(50)  NULL,
  mode         VARCHAR(16)  NOT NULL DEFAULT 'learn' COMMENT 'learn 学习模式 / sandbox 沙盘模式'
    ,
  quarters     INT          NOT NULL DEFAULT 14 COMMENT '完成季度数',
  total_score  DECIMAL(6,2) NULL COMMENT '综合评分',
  radar        JSON         NULL COMMENT '五维评分雷达',
  kpi_series   JSON         NULL COMMENT '逐季度指标序列',
  achievements JSON         NULL COMMENT '获得成就',
  policy_count INT          NOT NULL DEFAULT 0 COMMENT '执行政策数',
  quiz_correct INT          NOT NULL DEFAULT 0 COMMENT '学习模式答题正确数',
  quiz_total   INT          NOT NULL DEFAULT 0 COMMENT '学习模式答题总数',
  started_at   DATETIME     NULL,
  finished_at  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  KEY idx_user_time (user_id, finished_at)
) ENGINE=InnoDB COMMENT='游戏一对局记录表';

-- ------------------------------------------------------------
-- 表 15：学生画像（AI 生成的结构化画像，按用户唯一，重复生成覆盖）
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS student_profile (
  id           BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id      VARCHAR(64)  NOT NULL,
  radar        JSON         NULL COMMENT '六维能力雷达',
  dimensions   JSON         NULL COMMENT '细分维度（知识点/习惯/游戏/AI 使用）',
  tags         JSON         NULL COMMENT '画像标签',
  weak_nodes   JSON         NULL COMMENT '薄弱知识节点',
  narrative    TEXT         NULL COMMENT 'AI 生成的文字画像',
  summary      VARCHAR(200) NULL COMMENT '一句话画像摘要',
  source       VARCHAR(16)  NOT NULL DEFAULT 'rule' COMMENT 'rule 规则 / llm 大模型',
  model_name   VARCHAR(100) NULL,
  generated_at DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
  UNIQUE KEY uk_user (user_id),
  KEY idx_generated (generated_at)
) ENGINE=InnoDB COMMENT='学生画像表';

-- ------------------------------------------------------------
-- 表 16：学习建议（针对画像生成的可执行建议清单）
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS learning_advice (
  id           BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id      VARCHAR(64)  NOT NULL,
  items        JSON         NOT NULL COMMENT '建议条目数组（含优先级/依据/动作）',
  narrative    TEXT         NULL COMMENT 'AI 生成的整体说明',
  source       VARCHAR(16)  NOT NULL DEFAULT 'rule' COMMENT 'rule / llm / rl',
  policy_node  VARCHAR(40)  NULL COMMENT 'RL 策略推荐的下一知识节点',
  model_name   VARCHAR(100) NULL,
  generated_at DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  KEY idx_user_time (user_id, generated_at)
) ENGINE=InnoDB COMMENT='学习建议表';

-- ------------------------------------------------------------
-- 表 17：强化学习经验（DQN 的 (s,a,r,s') 转移，未作答时 reward 为空表示待回收）
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS rl_transition (
  id         BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id    VARCHAR(64)  NOT NULL,
  state      JSON         NOT NULL COMMENT '推荐时编码的学生状态（16 维）',
  action     INT          NOT NULL COMMENT '动作下标（知识图谱节点序号）',
  node_id    VARCHAR(40)  NOT NULL COMMENT '被推荐的知识节点 ID',
  reward     DECIMAL(7,4) NULL COMMENT '事后回收的奖励，NULL=尚未观测',
  next_state JSON         NULL COMMENT '作答后重新编码的状态',
  done       TINYINT(1)   NOT NULL DEFAULT 0,
  created_at DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  filled_at  DATETIME     NULL,
  KEY idx_user_time (user_id, created_at),
  KEY idx_pending (user_id, reward)
) ENGINE=InnoDB COMMENT='强化学习经验转移表';

-- ------------------------------------------------------------
-- 表 18：强化学习模型存档（DQN 权重 + 回放缓冲区，单行覆盖式保存）
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS rl_model (
  model_name   VARCHAR(40)  PRIMARY KEY COMMENT '模型名，如 islm-dqn-v1',
  state_dim    INT          NOT NULL,
  action_dim   INT          NOT NULL,
  weights      LONGTEXT     NOT NULL COMMENT 'DQN 序列化 JSON',
  train_steps  INT          NOT NULL DEFAULT 0,
  avg_reward   DECIMAL(7,4) NULL COMMENT '近期平均奖励（监控用）',
  meta         JSON         NULL COMMENT '版本、隐层结构、训练时间等',
  updated_at   DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB COMMENT='强化学习模型存档表';
```